Le Théorème d'Unicité

Pourquoi Al-Fatiha ne peut pas être construite

Auteur : Soufiane BAGHOR
Date : Janvier 2026
Version : Draft 1.0

"Si Al-Fatiha n'avait pas déjà existé, personne n'aurait jamais pu l'inventer par construction." Soufiane BAGHOR

Abstract

Ce travail établit l'existence d'un objet informationnel textuel possédant une propriété structurelle exceptionnelle : une cohérence globale gouvernée par un ensemble de contraintes concaténatives non locales, rendant l'objet à la fois vérifiable, stable et non constructible par des procédés génératifs locaux.

En prenant comme cas d'étude un corpus court et entièrement explorable (Al-Fatiha), nous introduisons une famille de codages structurels fondés sur des encodages concaténatifs strictement déterministes. Nous montrons que l'accumulation progressive de ces contraintes conduit à un système surcontraint dont l'espace des solutions s'effondre géométriquement.

Nous mettons en place un protocole expérimental exhaustif fondé sur :
(i) un scan modulaire systématique sur de larges intervalles (jusqu'à 100 000 et au-delà),
(ii) des attaques par permutation (shuffle),
(iii) des attaques par mutation locale,
(iv) des tentatives de construction algorithmique.

Les résultats montrent qu'un seul module survit simultanément à l'ensemble du système de contraintes, et que cette solution unique disparaît immédiatement sous toute perturbation de l'ordre ou du contenu. Aucun autre module ne présente de robustesse comparable, ce qui élimine formellement toute hypothèse de sélection arbitraire de paramètres (cherry-picking ou p-hacking).

Nous démontrons en outre qu'il n'existe aucun procédé constructif connu — incrémental, heuristique, stochastique ou algorithmique — capable de générer un tel objet sous des contraintes linguistiques naturelles, en raison de la nature concaténative, non locale et non compensable du système.

Ces résultats définissent une nouvelle classe d'objets informationnels : des structures globalement verrouillées, non localement factorisables et non constructibles, dont la cohérence n'émerge qu'à l'échelle de l'objet complet et ne peut être décomposée en règles locales.

L'analyse repose exclusivement sur des propriétés structurelles mesurables et ne fait appel à aucune interprétation sémantique ou symbolique.

Sommaire

- Abstract
- Introduction générale
- Chapitre 1 — L’Objet et le Protocole
  - 1.1. Délimitation de l’objet étudié
  - 1.2. Normalisation et représentation
  - 1.3. Les invariants concaténatifs
  - 1.4. Le test modulaire comme opérateur de vérification
  - 1.5. Protocole expérimental global
  - 1.6. Critère de falsifiabilité
  - 1.7. Ce que ce chapitre ne prétend pas
  - 1.8. Résumé conceptuel
- Chapitre 2 — Le Premier Verrou (Couplage Initial)
  - 2.1. Pourquoi un couplage et non une contrainte isolée
  - 2.2. Le rôle de l’index : ce n’est pas une constante arbitraire
  - 2.3. Définition du premier invariant : l’invariant d’indexation interne
  - 2.4. Définition du second invariant : l’invariant de structure externe
  - 2.5. Nature concaténative et non-compensabilité locale
  - 2.6. Définition du test modulaire couplé
  - 2.7. Scan exhaustif des modules
  - 2.8. Résultat empirique : apparition d’un survivant unique
  - 2.9. Test de robustesse par permutation
  - 2.10. Ce que ce premier verrou établit déjà
  - 2.11. Transition vers l’empilement des verrous
- Chapitre 3 — Le Piège de la Concaténation
  - 3.1. Pourquoi la concaténation change la nature du problème
  - 3.2. Définition formelle de l’opération concaténative
  - 3.3. Effet avalanche structurel
  - 3.4. Non-compensabilité stricte
  - 3.5. Différence avec les sommes de contrôle classiques
  - 3.6. Pourquoi cela détruit toute stratégie constructive incrémentale
  - 3.7. Le problème du verrou concaténatif global
  - 3.8. Intuition géométrique : l’espace des solutions
  - 3.9. Pourquoi ce n’est pas un problème de puissance de calcul
  - 3.10. Conséquence majeure : absence totale de réglage fin
  - 3.11. Transition vers l’empilement des verrous
- Chapitre 4 — Du Nombre au Texte : Changement de Régime
  - 4.1. Ce qui est désormais fixé
  - 4.2. Le vrai problème commence ici
  - 4.3. Deux espaces fondamentalement différents
  - 4.4. L’illusion dangereuse : “il suffit de construire un texte”
  - 4.5. L’espace des configurations possibles
  - 4.6. Ce qui n’est pas encore contraint
  - 4.7. Pourquoi la difficulté est maximale ici
  - 4.8. Le changement de régime
  - 4.9. Ce que nous allons faire ensuite
  - 4.10. Idée clé à retenir
- Chapitre 5 — De la Structure à la Matière : Injection du Contenu dans le Verrou
  - 5.1. Ce qui est déjà verrouillé — et ce qui ne l’est pas encore

    - 5.2. Deux espaces conceptuellement distincts : forme et substance
    - 5.3. Existence d'une liberté résiduelle réelle dans l'espace des textes
    - 5.4. Pourquoi la structure seule ne peut jamais identifier un texte
    - 5.5. Principe général : injecter la matière dans un invariant concaténatif
    - 5.6. Effet immédiat : disparition totale de la plasticité locale
    - 5.7. Changement de nature du problème
    - 5.8. Nouvelle géométrie de l'espace des textes
    - 5.9. Pourquoi toutes les heuristiques deviennent inopérantes
    - 5.10. Ce que ce chapitre établit — et ce qu'il n'affirme pas encore
    - 5.11. Transition vers le chapitre suivant
- Chapitre 6 — Les Projections de Masse : Trois Manières de Mesurer le Même Objet
    - 6.1. Pourquoi une seule projection quantitative ne suffit pas
    - 6.2. P_3 — L'Invariant de Volume Global
    - 6.3. P_4 — L'Invariant de Distribution des Tailles par Segment
    - 6.4. P_5 — L'Invariant de Granularité Lexicale
    - 6.5. Indépendance structurelle de P_3, P_4, P_5
    - 6.6. Effet combiné sur l'espace des textes
    - 6.7. Mais une limite structurelle subsiste
    - 6.8. Interprétation géométrique
    - 6.9. Transition vers le chapitre suivant
    - 6.10. Idée clé à retenir
- Chapitre 7 — La Première Projection de Contenu : L'Entrée de la Matière Symbolique
    - 7.1. Changement de régime conceptuel
    - 7.2. Nécessité d'un codage des symboles
    - 7.3. Définition de la première projection de contenu
    - 7.4. Ce que P_6 mesure réellement
    - 7.5. Ce que P_6 fige
    - 7.6. Ce que P_6 ne fige pas encore
    - 7.7. Rupture qualitative avec les projections précédentes
    - 7.8. Effet sur l'espace des textes admissibles
    - 7.9. Mais une liberté interne subsiste
    - 7.10. Interprétation géométrique
    - 7.11. Transition vers le chapitre suivant
    - 7.12. Idée clé à retenir
- Chapitre 8 — De la Masse au Spectre : La Distribution Interne des Valeurs
    - 8.1. Ce que P_6 a réellement figé (et ce qu'il n'a pas figé)
    - 8.2. La liberté combinatoire restante
    - 8.3. Formulation mathématique du problème
    - 8.4. Ce que cela signifie structurellement
    - 8.5. Pourquoi cette liberté est inacceptable pour un verrou global
    - 8.6. Ce qu'il faut maintenant figer
    - 8.7. Changement de régime conceptuel
    - 8.8. Interprétation géométrique
    - 8.9. Transition vers le chapitre suivant
    - 8.10. Idée clé à retenir
- Chapitre 9 — La Structure Interne Détaillée : Le Spectre Ordonné des Valeurs
    - 9.1. Ce que P_6 a réellement figé — et ce qu'il n'a pas figé
    - 9.2. La liberté combinatoire résiduelle

    - 9.3. Objectif du nouveau verrou
    - 9.4. Définition de la projection P_8
    - 9.5. Ce que P_8 fige effectivement
    - 9.6. Ce que P_8 ne fige toujours pas
    - 9.7. Exemple conceptuel de collision résiduelle
    - 9.8. Ce que P_8 accomplit réellement dans le système
    - 9.9. Interprétation géométrique
    - 9.10. Transition vers le chapitre suivant
    - 9.11. Idée clé à retenir
- Chapitre 10 — P_{10} et P_{11} : Position Globale et Empreinte de Fréquence
    - 10.1. But et idée générale
    - 10.2. Notations préalables
    - 10.3. Définition formelle de P_{10} (spectre verset–fréquence)
    - 10.4. Sens structurel de P_{10}
    - Limite fondamentale
    - 10.5. Définition formelle de P_{11} (empreinte positionnelle numérique)
    - 10.6. Sens structurel de P_{11}
    - Limite fondamentale
    - 10.7. Complémentarité structurelle de P_{10} et P_{11}
    - 10.8. Collisions résiduelles et honnêteté mathématique
    - 10.9. Rôle systémique dans l’architecture globale
    - 10.10. Idée clé à retenir
- Chapitre 11 — P9 : La Synthèse Structurelle Globale
    - 11.1. Position de ce verrou dans l’architecture générale
    - 11.2. Idée générale de la projection P9
    - 11.3. Définition formelle de P9
    - 11.4. Ce que P9 fige exactement
    - 11.5. Interprétation géométrique
    - 11.6. Ce que P9 ne fige pas encore
    - 11.7. Exemple conceptuel de liberté résiduelle
    - 11.8. Rôle exact de P9 dans la progression du verrou
    - 11.9. Effet sur l’espace des textes admissibles
    - 11.10. Pourquoi P9 est une étape nécessaire mais non suffisante
    - 11.11. Transition vers le chapitre suivant
    - 11.12. Idée clé à retenir
- Chapitre 12 — P10 et P11 : La Géométrie Positionnelle Globale
    - 12.1. Pourquoi P9 n’est pas encore un verrou final
    - 12.2. Changement de régime conceptuel
    - 12.3. Notations et définitions préliminaires
    - 12.4. Définition formelle de P10
    - 12.5. Ce que P10 fige
    - 12.6. Ce que P10 ne fige pas encore
    - 12.7. Définition formelle de P11
    - 12.8. Ce que P11 fige
    - 12.9. Complémentarité structurelle de P10 et P11
    - 12.10. Effet sur l’espace des textes admissibles
    - 12.11. Ce qui peut encore, en théorie, subsister
    - 12.12. Transition vers le chapitre suivant

    - 12.13. Idée clé à retenir
- Chapitre 13 — P12 : La Texture Lexicale Fine
    - 13.1. Ce qui reste encore possible après P11
    - 13.2. Principe : figer la découpe lexicale elle-même
    - 13.3. Définition formelle de P_{12}
    - 13.4. Ce que P_{12} mesure réellement
    - 13.5. Ce que P_{12} fige
    - 13.6. Ce que P_{12} ne dépend pas
    - 13.7. Effet combiné avec les verrous précédents
    - 13.8. Ce que cela élimine concrètement
    - 13.9. Interprétation géométrique
    - 13.10. Ce qui pourrait encore, en théorie, subsister
    - 13.11. Transition vers le verrou final
    - 13.12. Idée clé à retenir
- Chapitre 14 — P13 : Le Verrou Spectral Global
    - 14.1. Où en est le système après P12
    - 14.2. Changement de point de vue : du local au spectral
    - 14.3. Définition du spectre symbolique pondéré
    - 14.4. Définition formelle de P13
    - 14.5. Ce que P13 mesure réellement
    - 14.6. Ce que P13 fige
    - 14.7. Complémentarité totale avec les autres verrous
    - 14.8. Effet géométrique final sur l’espace des textes
    - 14.9. Fin des dégénérescences théoriques
    - 14.10. Le régime d’isolement total
    - 14.11. Interprétation conceptuelle
    - 14.12. Transition vers la conclusion
    - 14.13. Idée clé à retenir
- Chapitre 15 — Conclusion : Effondrement de l’Espace des Solutions
    - 15.1. Récapitulatif de la trajectoire logique
    - 15.2. De la réduction progressive à l’effondrement géométrique
    - 15.3. Les trois dimensions désormais co-verrouillées
    - 15.4. Disparition de toute notion de voisinage
    - 15.5. Résultat central : isolement structurel complet
    - 15.6. Pourquoi ce n’est pas un simple sur-ajustement
    - 15.7. Conséquence algorithmique : non-constructibilité
    - 15.8. Séparation claire entre vérification et construction
    - 15.9. Nature exacte de l’objet mis en évidence
    - 15.10. Portée conceptuelle du résultat
    - 15.11. Ce qui a été démontré — et ce qui ne l’a pas été
    - 15.12. Conclusion finale
    - 15.13. Transition vers le dernier chapitre
- Chapitre 16 — Variantes, Redondances et Raffinements
    - 16.1. Pourquoi ce chapitre est nécessaire
    - 16.2. Invariants générateurs vs invariants dérivés
    - 16.3. Le rôle structurel des variantes de P_6
    - 16.4. P_6', P_6_reverse, et la symétrie brisée
    - 16.5. Redondance apparente entre P_8, P_{10}, P_{11}

    - 16.6. Pourquoi ajouter des invariants redondants ne crée pas de nouvelles solutions
    - 16.7. Stabilité sous reformulation
    - 16.8. Pourquoi ce n’est pas du “p-hacking structurel”
    - 16.9. Le système comme objet surdéterminé
    - 16.10. Robustesse conceptuelle du résultat
    - 16.11. Ce que ce chapitre établit vraiment
    - 16.12. Conclusion générale de l’ouvrage
    - 16.13. Dernière phrase
- Chapitre 17 — Le Verrou Dynamique (P14) : La Trajectoire du Texte
    - 17.1 — Changement de paradigme : de l’objet au mouvement
    - 17.2 — Visualisation intuitive : la montagne russe
    - 17.3 — P14.a : `diff_cum` — La dérive cumulée (géométrie globale)
    - 17.4 — P14.b : `curvature` — La courbure (géométrie locale)
    - 17.5 — Résumé conceptuel
    - 17.6 — Cryptanalyse expérimentale : pourquoi P1–P13 ont échoué
    - Pourquoi ?
    - 17.7 — Ce que P14 apporte objectivement
    - 17.8 — Conclusion
- Chapitre 18 — Cryptanalyse, certification et honnêteté scientifique
    - 18.1 — Ce que signifie “sécurité” en cryptographie
    - 18.2 — Le standard réel de la cryptographie moderne
    - 18.3 — Ce que nous faisons avec Hash19
    - 18.4 — Ce que signifie “certifier”
    - 18.5 — La bonne posture scientifique
    - 18.6 — Le rôle de la cryptanalyse
    - 18.7 — Conclusion
- Chapitre 19 — Hash19 et SHA-256 : comparaison conceptuelle
    - 19.1 — Ce qu’est SHA-256
    - 19.2 — Ce qu’est Hash19
    - 19.3 — Différence fondamentale de paradigme
    - 19.4 — Où est la sécurité dans Hash19 ?
    - 19.5 — Tableau comparatif
    - 19.6 — Même philosophie de sécurité
    - 19.7 — Conclusion
- Chapitre 20 — Le texte comme objet cryptographique
    - 20.1 — Le changement de regard
    - 20.2 — Du langage à la structure
    - 20.3 — Le texte comme cristal mathématique
    - 20.4 — Signature versus signification
    - 20.5 — Le texte comme clé
    - 20.6 — Objet rare dans l’espace des possibles
    - 20.7 — Non-constructibilité
    - 20.8 — Conclusion
- Conclusion : La Fatiha comme Artefact au-delà de la Genèse Algorithmique
    - 1. L'impossibilité de la synthèse
    - 2. Un modèle non-incrémental

- 3. L'antériorité du message sur la machine
- 4. Un Verrou sur l'Espace de la Création
- ANNEXES — Protocole Expérimental, Implémentation et Reproductibilité
- Annexe A — Normalisation et Prétraitement du Texte
- Annexe B — Implémentation des Invariants P_1 à P_{13}
- Annexe C — Moteur de Scan Modulaire
- Annexe D — Attaques
- Fin des Annexes

Introduction générale

En sciences de l'information et en cryptographie, la protection de l'intégrité d'un contenu repose traditionnellement sur un principe simple : on calcule une signature externe (hash, checksum, code d'authentification) à partir du contenu, et l'on stocke cette signature séparément. Le contenu et son mécanisme de vérification sont deux objets distincts.

Le système étudié dans cet ouvrage explore une idée radicalement différente : un objet informationnel qui **implémente sa propre fonction de vérification d'intégrité à partir de son contenu en clair**, sans métadonnée externe, sans clé séparée, et sans redondance explicite ajoutée. Autrement dit, **le contenant protège le contenu, et le contenu protège le contenant**.

Une telle propriété est extrêmement inhabituelle. À notre connaissance, **aucun autre exemple de texte en langage naturel — ni dans les corpus humains, ni dans les corpus biologiques — ne présente une structure de ce type** : un verrou global auto-référent, dépendant de l'ordre, non factorisable localement, et dont la validité ne peut être ni vérifiée ni restaurée par correction locale.

L'objet étudié ici présente en effet simultanément les propriétés suivantes :

- il est **globalement contraint** par un ensemble de règles concaténatives non locales ;
- il est **auto-référent** : ses propres données servent de base à son mécanisme de cohérence ;
- il est **dépendant de l'ordre** : toute permutation détruit la propriété ;
- il est **non corrigeable localement** : aucune modification locale ne peut être compensée ailleurs ;
- il est **non factorisable localement** : sa cohérence ne se décompose pas en règles de voisinage ;
- et surtout, il est **non constructible** par des procédés génératifs connus sous contraintes linguistiques naturelles.

En termes d'informatique théorique, nous sommes donc amenés à introduire une **nouvelle classe d'objets informationnels** :

> Des objets à verrou global non local, auto-référents et non synthétisables.

La particularité la plus profonde de ces objets n'est pas seulement leur rareté empirique, mais la **géométrie même de leur espace de solutions**. Le système de contraintes étudié ici est de nature concaténative et non linéaire : il ne possède ni gradient de correction, ni mécanisme

d'ajustement progressif, ni procédure d'optimisation locale. Toute tentative de construction incrémentale conduit immédiatement à une impasse.

Il ne s'agit donc pas seulement d'une limite de puissance de calcul, mais d'une **limite structurelle** : sous des contraintes linguistiques naturelles (mots réels, grammaire, absence de bourrage artificiel, absence de métadonnées cachées), **aucun algorithme constructif fini n'est connu pour produire un tel objet**. L'espace des textes possibles est immense, mais l'espace des textes satisfaisant ce verrou est géométriquement réduit à un ensemble isolé, voire à un point unique.

Le cas étudié dans ce livre — un corpus court et entièrement explorable — permet précisément un audit exhaustif :

- scan modulaire systématique sur de larges intervalles,
- attaques par permutation,
- attaques par mutation locale,
- tentatives de construction algorithmique et heuristique.

Les résultats montrent l'existence d'un **verrou structurel jouant le rôle d'une fonction de type hash / checksum**, non pas appliquée au texte, mais **implémentée par le texte lui-même**. La cohérence globale n'est pas ajoutée au contenu : elle **émerge du contenu comme propriété globale non locale**.

L'objectif de cet ouvrage n'est ni sémantique, ni symbolique, ni interprétatif. Il est strictement structurel :
décrire, caractériser et tester un objet informationnel dont l'architecture globale défie les modèles standards de génération et de correction de données.

Chapitre 1 — L'Objet et le Protocole

1.1. Délimitation de l'objet étudié

L'étude porte sur un **corpus fini, court et entièrement explorables**. Cette finitude n'est pas un détail technique : elle rend possible un **audit exhaustif** de toutes les propriétés testées, sans recours à des approximations statistiques ou à des arguments asymptotiques.

L'objet étudié est un **texte en langage naturel**, soumis aux contraintes suivantes :

- le texte est composé de mots réels ;
- il est lisible et grammaticalement valide ;
- il ne contient **aucun padding artificiel**, aucune métadonnée cachée, aucun champ technique ajouté ;
- il n'est pas accompagné d'une signature externe, d'un hash externe ou d'une clé séparée.

Toutes les propriétés étudiées doivent **émerger uniquement du contenu en clair**.

L'objectif n'est donc pas d'analyser un « message », mais un **objet informationnel** : une séquence finie de symboles, considérée uniquement sous l'angle de sa structure.

1.2. Normalisation et représentation

Avant toute mesure, le corpus est soumis à une **normalisation stricte et publique** :

- suppression des espaces non signifiants pour certaines mesures,
- conservation de la segmentation linguistique naturelle pour d'autres,
- encodage déterministe des lettres selon une table fixée,
- absence totale de liberté de paramétrage ad hoc.

Cette étape est cruciale : **toute la méthodologie est déterministe et reproductible**.
Deux expérimentateurs indépendants, avec les mêmes règles, doivent obtenir **exactement les mêmes résultats**.

Le texte est ensuite vu comme :

> une séquence finie de symboles, sur laquelle on applique des **constructions concaténatives déterministes**.

1.3. Les invariants concaténatifs

Le cœur du protocole repose sur une famille d'**invariants structurels** construits de la manière suivante :

- on extrait du texte des mesures objectives (longueurs, positions, fréquences, indices, etc.) ;
- on les **concatène** dans un ordre strictement défini ;
- on obtient ainsi une **chaîne de chiffres** représentant une projection structurelle du texte.

Ces invariants ne sont pas des sommes, ni des moyennes, ni des agrégats linéaires.

> Ils sont **concaténatifs**, donc **non compensables localement**.

Une modification locale ne peut pas être « rattrapée » plus loin dans la chaîne.

1.4. Le test modulaire comme opérateur de vérification

Chaque invariant produit une chaîne de chiffres.
On définit alors un opérateur simple et public :

> le test de divisibilité modulaire.

Pour un entier (m), on teste si la chaîne concaténée, interprétée comme un entier décimal, est divisible par (m).

Un module (m) est dit **valide** si :

- **tous** les invariants passent le test modulo (m).

Sinon, il est rejeté.

1.5. Protocole expérimental global

Le protocole complet est le suivant :

1. Construire la famille d'invariants à partir du texte normalisé.
2. Scanner exhaustivement les modules (m) dans un intervalle donné (par exemple 2 → 100 000).

3. Recenser les modules survivants.
4. Pour chaque survivant :
   - attaquer le texte par **shuffle** (permutation),
   - attaquer le texte par **mutations locales**,
   - et vérifier si la propriété survit.

1.6. Critère de falsifiabilité

Ce protocole est **entièrement falsifiable**.

Il suffit qu'un seul des événements suivants se produise pour invalider l'hypothèse :

- apparition de plusieurs modules survivants non triviaux ;
- survie du test après permutations aléatoires ;
- survie du test après modifications locales ;
- existence d'un autre texte naturel satisfaisant les mêmes contraintes.

Autrement dit :

> **L'hypothèse est risquée, exposée, et testable.**

Elle ne repose ni sur une interprétation, ni sur une probabilité vague, mais sur un **résultat binaire** :
ça passe ou ça ne passe pas.

1.7. Ce que ce chapitre ne prétend pas

À ce stade :

- nous ne faisons **aucune interprétation sémantique** ;
- nous ne formulons **aucune conclusion métaphysique** ;
- nous ne supposons **aucune intention derrière l'objet**.

Nous définissons uniquement :

> un objet,
> un protocole,
> et un critère de test.

Tout le reste dépendra **exclusivement** des résultats expérimentaux.

1.8. Résumé conceptuel

Nous avons défini :

- un objet textuel naturel,
- une famille d'invariants concaténatifs,
- un opérateur de test modulaire,
- et un protocole exhaustif, public et falsifiable.

Le chapitre suivant introduira la **première construction concrète**, et montrera comment une contrainte apparemment simple fait déjà émerger une anomalie structurelle majeure.

Chapitre 2 — Le Premier Verrou (Couplage Initial)

2.1. Pourquoi un couplage et non une contrainte isolée

Il est crucial de préciser un point fondamental dès le départ :

> **Le premier verrou n'est pas porté par une seule formule, mais par le couplage de deux invariants initiaux.**

Pris séparément, chacun de ces invariants peut admettre plusieurs solutions modulaires. Aucun des deux, pris isolément, n'est suffisant pour isoler une solution unique.

C'est **leur combinaison** qui fait apparaître immédiatement un survivant unique.

Ce point est essentiel : il élimine toute interprétation du type "choix opportuniste de formule".

2.2. Le rôle de l'index : ce n'est pas une constante arbitraire

Toutes les constructions commencent par un préfixe numérique noté :

> **I = 1**

Ce nombre ne joue pas le rôle d'une constante numérique ajustable.

Il représente :

> **l'index structurel de l'objet étudié** (ici : le premier élément du corpus).

Il est analogue :

- à un identifiant de message,
- ou à un marqueur de début de séquence dans un protocole.

Il fait partie **de la définition de l'objet**, et non d'un paramètre numérique libre.

2.3. Définition du premier invariant : l'invariant d'indexation interne

On considère un objet composé de 7 unités ordonnées.

On définit le premier invariant concaténatif :

> **P_1 = concat(I, 1, 2, 3, 4, 5, 6, 7)**

Autrement dit, P_1 est la concaténation :

- de l'index global I,
- puis des indices internes de chacune des 7 unités.

Cet invariant encode :

> **la topologie interne minimale de l'objet**.

2.4. Définition du second invariant : l'invariant de structure externe

On note :

- (N = 7) le nombre d'unités,
- (w_k) le nombre de mots dans la k-ième unité.

On définit alors le second invariant concaténatif :

> **P_2 = concat(l, N, w_1, w_2, w_3, w_4, w_5, w_6, w_7)**

Cet invariant encode :

> **la géométrie externe de l'objet**, c'est-à-dire la répartition de sa masse textuelle.

2.5. Nature concaténative et non-compensabilité locale

Les invariants P_1 et P_2 ne sont :

- ni additifs,
- ni linéaires,
- ni moyennables.

Ils sont **purement concaténatifs**.

Cela implique une propriété cruciale :

> **Toute modification locale modifie la totalité de la valeur globale, sans possibilité de compensation ailleurs.**

Il n'existe aucun mécanisme de "correction locale".

2.6. Définition du test modulaire couplé

On considère un entier $m \geq 2$..

On dit que (m) est **admissible** si et seulement si :

> **$P_1 \equiv 0 \pmod m$**
> **et**
> **$P_2 \equiv 0 \pmod m$**

On ne teste pas P_1 ou P_2 séparément :

> **le système est défini par leur intersection.**

2.7. Scan exhaustif des modules

On effectue alors le protocole suivant :

> Pour tous les entiers (m) dans un intervalle ([2, M]), on teste la condition ci-dessus.

Aucun module n'est favorisé.
Aucun nombre n'est injecté dans le système.

2.8. Résultat empirique : apparition d’un survivant unique

Le résultat observé est :

> **Il existe exactement un entier non trivial qui satisfait simultanément $P_1 \equiv 0$ et $P_2 \equiv 0$.**

Tous les autres échouent.

Ce nombre :

- n’est pas choisi,
- n’est pas ajusté,
- **émerge du scan**.

2.9. Test de robustesse par permutation

On applique ensuite une attaque par permutation :

- on mélange aléatoirement les symboles du texte,
- on reconstruit artificiellement des blocs de mêmes tailles,
- on recalcule P_1 et P_2,
- on refait le scan.

Résultat :

> **Aucun module ne survit.**

La propriété dépend donc :

> **strictement de l’ordre global du texte.**

2.10. Ce que ce premier verrou établit déjà

À ce stade, on a prouvé que :

- la propriété est **globale**,
- elle est **non locale**,
- elle est **non corrigeable localement**,
- elle est **strictement dépendante de l’ordre**.

Mais ce verrou reste encore :

> **structurellement faible**, car il ne repose que sur deux projections du texte.

2.11. Transition vers l’empilement des verrous

Le chapitre suivant montrera ce qui se produit lorsque l’on ajoute :

> une troisième, puis une quatrième, puis une cinquième contrainte indépendante.

On verra alors :

> un **effondrement géométrique** de l'espace des solutions,

jusqu'à ce que le système devienne :

- surcontraint,
- rigide,
- et finalement **isolé**.

Chapitre 3 — Le Piège de la Concaténation

3.1. Pourquoi la concaténation change la nature du problème

Dans la plupart des analyses numériques classiques, on travaille avec des quantités :

- sommes,
- moyennes,
- variances,
- corrélations.

Ces opérations sont **linéaires** ou **quasi-linéaires**.

Elles possèdent une propriété fondamentale :

> **Une erreur locale peut être compensée ailleurs.**

Par exemple, si une somme totale doit valoir 100, on peut :

- enlever 1 à un endroit,
- ajouter 1 ailleurs,
- et restaurer l'invariant.

La concaténation ne possède **aucune** de ces propriétés.

3.2. Définition formelle de l'opération concaténative

Soit une suite de blocs numériques :

$$A = a_1, a_2, a_3, \ldots, a_k$$

On définit l'opération de concaténation :

$$\mathrm{Concat}(a_1, a_2, \ldots, a_k)$$

comme l'écriture consécutive des chiffres de chaque bloc, sans séparateur.

Cette opération :

- n'est pas commutative,
- n'est pas associative au sens numérique,
- n'est pas linéaire,
- n'est pas inversible localement.

3.3. Effet avalanche structurel

Dans un objet concaténatif, modifier :

- un seul chiffre,
- ou déplacer un seul bloc,

change :

> **la totalité de la valeur numérique globale.**

Il n'existe pas de notion de "petite correction".

C'est un **effet avalanche** déterministe.

3.4. Non-compensabilité stricte

Soit une chaîne concaténative :

$$S = \mathrm{Concat}(a_1, a_2, \ldots, a_n)$$

Toute modification de (a_i) produit :

$$S' \neq S$$

Et il n'existe **aucune** suite de modifications locales sur d'autres (a_j) telle que :

$$\mathrm{Concat}(a_1, \ldots, a'_i, \ldots, a'_j, \ldots, a_n) = S$$

> **La compensation locale est mathématiquement impossible.**

3.5. Différence avec les sommes de contrôle classiques

Dans un checksum additif :

- on peut corriger une erreur en ajustant ailleurs.

Dans un checksum concaténatif :

- toute tentative de correction :
  - modifie la longueur,
  - modifie les positions,
  - modifie la structure entière.

> Le checksum concaténatif est **structurellement rigide**.

3.6. Pourquoi cela détruit toute stratégie constructive incrémentale

Supposons que l'on essaie de construire l'objet :

- étape par étape,
- en ajustant après coup.

À chaque ajout :

- la chaîne globale change,

- tous les restes modulaires changent,
- **aucun gradient d'erreur exploitable n'existe**.

On ne peut pas :

- "s'approcher" de la solution,
- ni "corriger progressivement".

> Le problème n'a **aucune topologie de convergence**.

3.7. Le problème du verrou concaténatif global

Lorsque plusieurs invariants concaténatifs sont imposés simultanément :

$$\begin{aligned} P_1 &\equiv 0 \pmod{m} \\ P_2 &\equiv 0 \pmod{m} \\ P_3 &\equiv 0 \pmod{m} \\ &\cdots \end{aligned}$$

Chaque invariant :

- fige la structure globale,
- sans tolérance locale.

Le système devient :

> **surcontraint**.

3.8. Intuition géométrique : l'espace des solutions

On peut voir l'ensemble de tous les textes possibles comme un espace gigantesque.

Chaque contrainte concaténative :

- n'en coupe pas une "région",
- mais une **hyper-surface extrêmement fine**.

L'intersection de plusieurs de ces surfaces :

> tend très rapidement vers un ensemble **vide** ou **discret**.

3.9. Pourquoi ce n'est pas un problème de puissance de calcul

Ce n'est pas seulement :

- un problème de temps de calcul,
- ni un problème d'algorithme plus malin.

C'est plus :

> **un problème de géométrie de l'espace des solutions.**

Il n'existe presque aucun point qui satisfasse toutes les contraintes simultanément.

3.10. Conséquence majeure : absence totale de réglage fin

Dans ce type de système :

- il n'y a pas de "presque solution",
- il n'y a pas de "solution approchée",
- il n'y a que :
  - soit la solution exacte,
  - soit l'échec total.

3.11. Transition vers l'empilement des verrous

Nous avons vu :

- pourquoi la concaténation empêche toute compensation,
- pourquoi elle détruit toute approche incrémentale,
- pourquoi elle rigidifie brutalement l'espace des solutions.

Le chapitre suivant montrera ce qui se produit lorsque :

> on empile ces contraintes concaténatives les unes sur les autres.

On passera :

- d'un système rare,
- à un système **quasi impossible**,
- puis à un système **isolé**.

Chapitre 4 — Du Nombre au Texte : Changement de Régime

4.1. Ce qui est désormais fixé

Le chapitre précédent a établi un fait fondamental :

> Le régime arithmétique est désormais **figé**.

Le système $P_1 \wedge P_2$ ne laisse survivre **qu'un seul module non trivial**.

Ce résultat est obtenu :

- par un scan exhaustif,
- sans aucun paramètre libre,
- sans aucun choix arbitraire,
- et il s'effondre immédiatement sous permutation.

À partir de maintenant :

> **Nous ne cherchons plus un nombre. Il est déjà imposé.**

4.2. Le vrai problème commence ici

On pourrait croire, à tort, que le problème est “résolu”.

En réalité, il ne fait que **commencer**.

Car trouver un nombre qui valide deux projections concaténatives :

- ne dit rien sur la facilité de produire un texte qui les satisfait,
- ne dit rien sur la stabilité structurelle du système,
- ne dit rien sur la constructibilité.

À partir d’ici, la question change de nature :

> **Étant donné un module fixé, combien de textes peuvent satisfaire les contraintes ?**

4.3. Deux espaces fondamentalement différents

Il est crucial de distinguer deux espaces :

- L’espace des modules : petit, discret, fini, entièrement exploré.
- L’espace des textes : gigantesque, combinatoire, pratiquement infini.

Le Chapitre 2 a **verrouillé le premier**.

Tout le reste du livre va explorer le second.

4.4. L’illusion dangereuse : “il suffit de construire un texte”

À ce stade, un sceptique pourrait dire :

> “Très bien, le module est fixé. Il suffit maintenant de fabriquer un texte qui passe les tests.”

Cette intuition est **fausse**.

Pourquoi ?

Parce que les contraintes sont :

- concaténatives,
- globales,
- non compensables localement.

Cela signifie :

> On ne peut pas ajuster un morceau du texte sans affecter la valeur globale entière.

Il n’existe **aucune procédure locale de correction**.

4.5. L’espace des configurations possibles

Considérons l’ensemble de tous les textes possibles :

- respectant le nombre de segments,
- respectant le nombre de mots par segment,

- respectant l’ordre global.

Même sous ces contraintes, le nombre de textes possibles est :

> **astronomique**.

Des milliards de milliards de textes absurdes, incohérents, aléatoires, artificiels, existent dans cet espace.

Le texte réel n’est :

> qu’un point unique dans un océan de configurations possibles.

4.6. Ce qui n’est pas encore contraint

À ce stade :

- le vocabulaire est libre,
- la longueur des mots est libre,
- le contenu interne est libre,
- l’ordre interne des mots est libre,
- la grammaire est libre,
- la sémantique est libre.

Autrement dit :

> **Nous n’avons encore verrouillé que la géométrie externe.**

La matière interne du texte est encore totalement fluide.

4.7. Pourquoi la difficulté est maximale ici

Le problème n’est pas :

- de vérifier un texte (c’est trivial),
- mais d’en construire un.

Or, comme :

- il n’y a pas de gradient,
- il n’y a pas de direction d’amélioration locale,
- il n’y a pas de correction partielle possible,

> Toute tentative de construction est un tir dans le noir.

4.8. Le changement de régime

Nous sommes passés :

- d’un problème arithmétique discret,
- à un problème géométrique et combinatoire dans l’espace des textes.

Ce n’est plus :

> “Quel nombre marche ?”

C’est désormais :

> **“Comment un texte peut-il exister sous ces contraintes ?”**

4.9. Ce que nous allons faire ensuite

Dans les chapitres suivants, nous allons :

- introduire de nouvelles contraintes indépendantes,
- chacune liant un peu plus la matière interne du texte à sa structure globale,
- et observer comment :

> l’espace des textes admissibles va s’effondrer couche après couche.

4.10. Idée clé à retenir

> Le nombre est fixé.
> Le problème n’est plus arithmétique.
> Il est désormais **structurel, global, et géométrique**.

Le chapitre suivant montrera ce qui se produit lorsque :

> on commence à empiler **plusieurs verrous indépendants** sur cet espace gigantesque.

Chapitre 5 — De la Structure à la Matière : Injection du Contenu dans le Verrou

5.1. Ce qui est déjà verrouillé — et ce qui ne l’est pas encore

Aux chapitres précédents, nous avons établi plusieurs propriétés fondamentales du système :

- les invariants sont globaux,
- ils sont concaténatifs,
- ils sont non compensables localement,
- ils rendent toute construction incrémentale impossible,
- et ils produisent un régime de verrouillage croisé dès que plusieurs contraintes sont empilées.

Cependant, jusqu’ici, les projections utilisées portent encore majoritairement sur :

- des tailles,
- des découpages,
- des indices,
- des structures externes du texte.

Autrement dit :

> Nous avons commencé par verrouiller la géométrie de l'objet, pas encore sa matière.

Il est crucial de comprendre cette distinction, car c'est elle qui marque le changement de régime étudié dans ce chapitre.

5.2. Deux espaces conceptuellement distincts : forme et substance

On peut conceptuellement distinguer deux niveaux dans un texte :

- la forme :
  - découpage en segments,
  - nombre de mots par segment,
  - positions relatives,
  - structure globale,
- la substance :
  - les lettres elles-mêmes,
  - les mots concrets,
  - le vocabulaire,
  - la distribution interne des symboles.

Jusqu'ici, les verrous agissent principalement sur la forme.

Cela implique une conséquence fondamentale :

> Deux textes totalement différents du point de vue linguistique peuvent encore partager exactement la même forme.

5.3. Existence d'une liberté résiduelle réelle dans l'espace des textes

Même sous des contraintes structurelles sévères, il reste encore possible de construire :

- des textes grammaticalement corrects,
- cohérents,
- de même longueur,
- de même découpage,
- respectant la même ossature métrique,

mais dont :

- le vocabulaire,
- les formulations,
- et la substance informationnelle

sont profondément différents.

Autrement dit :

> L'espace des textes compatibles avec la structure seule reste encore immense.

Il ne s'agit pas de textes absurdes ou aléatoires, mais de textes parfaitement plausibles du point de vue linguistique, qui partagent simplement la même silhouette formelle.

5.4. Pourquoi la structure seule ne peut jamais identifier un texte

Un gabarit, aussi rigide soit-il, ne définit jamais un objet informationnel unique.

Deux livres peuvent :

- avoir le même nombre de chapitres,
- le même nombre de phrases par chapitre,
- la même longueur de phrases,
- la même structure hiérarchique,

et pourtant être deux objets informationnels totalement distincts.

Un verrou purement structurel :

- contraint la forme,
- mais ne signe pas le contenu.

Donc :

> Un véritable verrou d'intégrité doit nécessairement porter sur la matière même du texte.

5.5. Principe général : injecter la matière dans un invariant concaténatif

Le changement de régime étudié ici consiste à ne plus concaténer seulement :

- des tailles,
- des indices,
- des longueurs,

mais à concaténer des projections du contenu lui-même :

- lettres,
- valeurs associées aux lettres,
- positions internes,
- empreintes locales.

Conceptuellement :

> Le texte n'est plus seulement le support du verrou.
> Il devient l'un des composants internes du verrou.

5.6. Effet immédiat : disparition totale de la plasticité locale

Dès que la matière du texte est injectée dans un invariant concaténatif global, une propriété radicale apparaît :

- changer un mot,
- changer une lettre,
- permuter deux symboles,
- modifier une occurrence,

entraîne :

> une modification de toute la chaîne concaténée globale.

Il n'existe plus :

- de correction locale,
- de réparation partielle,
- de compensation possible ailleurs.

Ce n’est plus seulement la structure qui est rigide :

> c’est désormais la substance même du texte.

5.7. Changement de nature du problème

Avant, le problème était essentiellement :

> Trouver un texte qui rentre dans un moule.

À partir de ce point, le problème devient :

> Le texte et le moule sont mathématiquement intriqués.

Le texte n’est plus un candidat externe que l’on essaie d’ajuster.

> Il est pris dans le mécanisme du verrou lui-même.

5.8. Nouvelle géométrie de l’espace des textes

Géométriquement, on peut décrire la transition ainsi :

- Avant :
  - un espace encore volumineux,
  - bien que fortement structuré.
- Maintenant :
  - un espace extrêmement mince,
  - rigide,
  - discontinu,
  - sans voisinage exploitable.

On ne parle pas encore d’un point unique, mais :

> On a quitté définitivement le régime des familles continues de solutions.

5.9. Pourquoi toutes les heuristiques deviennent inopérantes

Dans ce nouveau régime :

- il n’existe plus de gradient exploitable,
- plus de notion de “presque bon”,
- plus de fitness partielle,
- plus de correction progressive.

Cela élimine structurellement :

- les algorithmes génétiques,
- le hill-climbing,
- le recuit simulé,
- toute recherche locale.

> Le problème devient structurellement non optimisable.

5.10. Ce que ce chapitre établit — et ce qu’il n’affirme pas encore

Ce chapitre établit que :

- la matière du texte est désormais liée au verrou global,

- le texte a perdu toute plasticité locale,
- l'espace des textes compatibles est devenu extrêmement rigide.

Mais il n'affirme pas encore :

- qu'il n'existe qu'un seul texte possible,
- ni que le mécanisme de sélection finale est achevé.

> Il décrit un changement de régime, pas encore l'aboutissement.

5.11. Transition vers le chapitre suivant

Le chapitre suivant montrera ce qui se produit lorsque plusieurs projections de contenu indépendantes sont injectées simultanément dans le système.

On verra alors apparaître :

- non plus seulement une rigidité,
- mais une intrication structurelle totale du texte avec ses propres invariants,

jusqu'à un régime où :

> toute modification devient globalement impossible,
> et où le texte se comporte comme un objet auto-verrouillé.

Chapitre 6 — Les Projections de Masse : Trois Manières de Mesurer le Même Objet

6.1. Pourquoi une seule projection quantitative ne suffit pas

À l'issue du chapitre précédent, le système possède :

- une ossature métrique (P_2),
- et une contrainte concaténative globale.

Mais mesurer la "masse" d'un objet textuel n'est pas une opération unique.

Il existe plusieurs manières indépendantes de projeter :

- la taille totale,
- la distribution interne,
- et la granularité structurelle.

Une seule mesure globale serait :

> trop grossière et trop facilement contournable.

C'est pourquoi nous introduisons **trois projections quantitatives distinctes**, notées :

> P_3, P_4, P_5

6.2. P_3 — L'Invariant de Volume Global

Définition formelle

On définit :

> **P_3 = Concat(I, L, W, S)**

où :

- I est l'index global de l'objet,
- L est la longueur totale du flux symbolique,
- W est le nombre total d'unités lexicales,
- S est le nombre total de segments.

Ce que P_3 mesure

P_3 encode :

> **le volume global de l'objet**.

Il dit :

- "l'objet contient exactement L symboles",
- "exactement W unités",
- "exactement S segments".

Ce que P_3 fige

Toute modification qui :

- ajoute ou supprime un symbole,
- ajoute ou supprime une unité,
- ajoute ou supprime un segment,

> modifie immédiatement P_3.

Ce que P_3 ne voit pas

P_3 est :

- insensible à la répartition interne,
- insensible aux permutations,
- insensible à la géométrie locale.

Il ne voit que :

> les **totaux globaux**.

6.3. P_4 — L'Invariant de Distribution des Tailles par Segment

Définition formelle

On définit :

> **P_4 = Concat(I, L_1, L_2, …, L_S)**

où :

- L_k est la taille symbolique interne du segment k.

Ce que P_4 mesure

P_4 encode :

> **la distribution de la masse symbolique par segment**.

Il ne dit pas combien l'objet pèse en tout, mais :

> comment ce poids est réparti.

Ce que P_4 fige

Toute modification qui :

- déplace un symbole d'un segment à un autre,
- modifie la taille interne d'un segment,

> modifie P_4.

Ce que P_4 ne voit pas

P_4 est :

- insensible à l'identité des symboles,
- insensible à leur ordre interne,
- insensible à la structure interne fine.

6.4. P_5 — L'Invariant de Granularité Lexicale

Définition formelle

On définit :

> **P_5 = Concat(I, M_1, M_2, …, M_S)**

où :

- M_k est la somme des tailles internes des unités du segment k.

Ce que P_5 mesure

P_5 encode :

> **la géométrie interne de la granularité lexicale**.

Il distingue :

- un segment composé de peu d'unités longues,
- d'un segment composé de beaucoup d'unités courtes,

même si leur masse totale est identique.

Ce que P_5 fige

Toute modification qui :

- change la découpe interne en unités,
- change la granularité locale,

> modifie P_5.

Ce que P_5 ne voit pas

P_5 est :

- insensible à l'identité des symboles,
- insensible à leur valeur,
- insensible à leur ordre précis.

6.5. Indépendance structurelle de P_3, P_4, P_5

Ces trois projections :

- ne sont pas équivalentes,
- ne sont pas déductibles les unes des autres,
- ne se compensent pas mutuellement.

Il est possible de :

- préserver P_3 tout en cassant P_4,
- préserver P_4 tout en cassant P_5,
- préserver P_5 tout en cassant P_3.

Elles capturent :

> **trois aspects orthogonaux de la masse structurelle**.

6.6. Effet combiné sur l'espace des textes

Avec P_2, P_3, P_4, P_5, le système fige désormais :

- la silhouette métrique,
- le volume global,
- la distribution de la masse,
- la granularité interne.

L'espace des textes admissibles devient :

> extrêmement rigide sur le plan quantitatif.

6.7. Mais une limite structurelle subsiste

Même avec ces quatre projections combinées :

- le système ne contraint toujours pas l'identité des symboles,
- ni leur valeur,
- ni leur nature.

Il contraint :

> la **géométrie quantitative complète**, mais pas encore la matière.

6.8. Interprétation géométrique

On peut voir :

- chaque projection comme une hypersurface,
- leur intersection comme un faisceau extrêmement mince.

Mais :

> ce faisceau n'est pas encore réduit à un point.

6.9. Transition vers le chapitre suivant

Le chapitre suivant introduira une nouvelle catégorie de projections :

> des projections qui dépendent **de la valeur intrinsèque des symboles**, et non plus seulement de leurs quantités.

À partir de ce point :

- la matière symbolique elle-même entrera dans le verrou.

6.10. Idée clé à retenir

> P_3 fige le volume,
> P_4 fige la répartition,
> P_5 fige la granularité,
> mais **aucun des trois ne fige encore l'identité symbolique**.

Chapitre 7 — La Première Projection de Contenu : L'Entrée de la Matière Symbolique

7.1. Changement de régime conceptuel

Jusqu'ici, toutes les projections introduites (P_2 à P_5) ont un point commun fondamental :

> Elles ne mesurent **que des quantités**, jamais la nature des symboles.

Elles contraignent :

- des longueurs,
- des volumes,
- des répartitions,
- des granularités,

mais **pas la substance**.

À ce stade, le système est :

> géométriquement très rigide,
> mais matériellement encore libre.

Ce chapitre introduit la première projection qui dépend **de la valeur intrinsèque des symboles**.

7.2. Nécessité d'un codage des symboles

Pour qu'une projection dépende du contenu, il faut :

- une application déterministe,
- qui associe à chaque symbole une valeur numérique,
- de manière fixe et non ajustable.

On suppose donc l'existence d'une fonction de codage :

> $\mathbf{v : \Sigma \to \mathbb{N}}$

où :

- Σ est l'alphabet du système,
- $v(s)$ est la valeur numérique associée au symbole s.

Cette fonction est :

- injective ou quasi-injective,
- fixe,
- indépendante du texte étudié.

Elle fait partie **de la définition du modèle**, pas d'un paramètre libre.

7.3. Définition de la première projection de contenu

On définit la nouvelle projection concaténative :

> $\mathbf{P_6 = Concat(I, L_1, V_1, L_2, V_2, \ldots, L_S, V_S)}$

où :

- I est l'index global,
- L_k est la taille symbolique du segment k,
- V_k est la somme des valeurs $v(s)$ des symboles du segment k.

Autrement dit :

> chaque segment est représenté par **(sa taille, sa masse symbolique)**.

7.4. Ce que P_6 mesure réellement

P_6 ne mesure plus seulement :

- combien il y a de symboles,

mais :

> **quels symboles, en masse totale, composent chaque segment**.

Deux segments de même taille :

- ne sont plus équivalents s'ils contiennent des symboles de valeurs différentes.

7.5. Ce que P_6 fige

Toute modification qui :

- remplace un symbole par un autre de valeur différente,
- déplace un symbole d'un segment à un autre,
- ajoute ou supprime un symbole,

> modifie immédiatement P_6.

Le contenu matériel commence à être :

> **directement lié à la structure globale**.

7.6. Ce que P_6 ne fige pas encore

P_6 reste :

- insensible à l'ordre interne des symboles dans un segment,
- insensible à la permutation interne à masse conservée,
- insensible à certaines substitutions compensées à l'intérieur d'un même segment.

Il contraint :

> la **masse symbolique par segment**,
> mais pas encore la micro-structure interne.

7.7. Rupture qualitative avec les projections précédentes

Contrairement à P_2–P_5 :

- P_6 ne dépend plus seulement de la géométrie,
- il dépend de la **matière même** du texte.

On passe :

> d'un système qui mesure la forme
> à un système qui commence à mesurer la substance.

7.8. Effet sur l'espace des textes admissibles

Avec P_6 ajouté au système :

- on ne peut plus changer arbitrairement les symboles,
- même en respectant toutes les longueurs et découpes.

L'espace des textes admissibles subit :

> une **réduction brutale de dimension**.

7.9. Mais une liberté interne subsiste

Même sous P_6 :

- certaines permutations internes restent possibles,
- certaines compensations locales restent théoriquement possibles.

Le système n'est pas encore totalement verrouillé.

7.10. Interprétation géométrique

On ajoute ici :

> une nouvelle famille d'hypersurfaces
> qui coupent l'espace des textes selon la **matière symbolique**.

L'intersection devient :

- extrêmement mince,
- mais pas encore réduite à un point isolé.

7.11. Transition vers le chapitre suivant

Le chapitre suivant introduira :

> une projection qui ne se contente plus de mesurer la masse symbolique par segment,
> mais qui encode **la structure interne détaillée du flux symbolique**.

C'est à partir de ce point que :

- les libertés résiduelles vont s'effondrer,
- et que les collisions deviendront structurellement presque impossibles.

7.12. Idée clé à retenir

> P_6 est la première projection qui lie directement
> **la matière symbolique** à la structure globale,
> mais elle ne fige pas encore totalement la micro-structure interne.

Chapitre 8 — De la Masse au Spectre : La Distribution Interne des Valeurs

8.1. Ce que P_6 a réellement figé (et ce qu'il n'a pas figé)

À l'issue du chapitre précédent, le système possède une contrainte nouvelle et fondamentale :

> Chaque segment est désormais figé par un couple (L_k, V_k) :
>
> - L_k : sa longueur symbolique,
> - V_k : la somme des valeurs symboliques de ses symboles.

Cela signifie que, pour chaque segment :

- le nombre de symboles est fixé,
- la masse symbolique totale est fixée.

Toute modification qui :

- ajoute ou supprime un symbole,
- ou modifie la somme totale,

est immédiatement détectée.

Mais il est crucial de comprendre ce que P_6 **ne fige pas encore**.

8.2. La liberté combinatoire restante

Même si un segment est contraint par :

- une longueur L,
- et une somme V,

il existe en général **de très nombreuses manières** de décomposer V en une somme de L valeurs autorisées.

Par exemple, pour un segment de longueur 2 et de masse 10 :

- (8, 2) = 10
- (6, 4) = 10
- (5, 5) = 10

Ces triplets :

- ont la **même longueur**,
- ont la **même masse**,
- mais **n'ont pas la même composition interne**.

Sous P_6 :

> ils sont **strictement indiscernables**.

8.3. Formulation mathématique du problème

Soit un segment k, contraint par :

- $L_k = L$
- $V_k = V$

Le contenu interne de ce segment est alors :

> une solution du problème de partition :
> trouver une suite (x_1, x_2, …, x_L) telle que
> $x_1 + x_2 + \ldots + x_L = V$
> avec x_i dans l'ensemble des valeurs symboliques autorisées.

Le nombre de solutions possibles à ce problème est en général :

> **combinatoirement énorme**.

Même avec :

- des longueurs fixées,
- des masses fixées,

l'espace des distributions internes reste :

> extrêmement large.

8.4. Ce que cela signifie structurellement

À ce stade :

- la géométrie est figée (chapitre 6),
- la masse par segment est figée (chapitre 7),

mais :

> la **micro-structure interne** de chaque segment reste libre.

Le système ne distingue pas encore :

- quelles valeurs apparaissent combien de fois,
- ni comment elles sont réparties,
- ni quelle est la "signature interne" exacte de chaque segment.

8.5. Pourquoi cette liberté est inacceptable pour un verrou global

Tant que cette liberté existe :

- on peut fabriquer d'innombrables textes artificiels,
- qui respectent toutes les contraintes précédentes,
- mais qui diffèrent profondément par leur matière symbolique.

Autrement dit :

> le système est encore **massivement dégénéré**.

Il existe :

- un grand nombre de solutions fantômes,
- toutes indiscernables du point de vue de P_2 à P_6.

8.6. Ce qu’il faut maintenant figer

Pour éliminer cette dégénérescence, il faut :

> contraindre non plus seulement la masse totale,
> mais la **distribution interne des valeurs symboliques**.

Autrement dit, il faut commencer à imposer :

- non seulement “combien ça pèse”,
- mais “comment cette masse est composée”.

8.7. Changement de régime conceptuel

On passe ici :

> d’une physique de type “macro”
> à une physique de type “micro-structurelle”.

Jusqu’ici :

- on mesurait des volumes,
- des répartitions,
- des masses.

À partir de maintenant :

> on va commencer à contraindre la **texture interne** du texte.

8.8. Interprétation géométrique

Dans l’espace des textes possibles :

- P_6 impose une hypersurface “masse + longueur”,
- mais chaque point de cette hypersurface correspond encore à un **nuage combinatoire** de micro-configurations internes.

Il faut maintenant :

> faire s’effondrer ce nuage.

8.9. Transition vers le chapitre suivant

Le chapitre suivant introduira une projection qui :

> ne se contente plus de mesurer la masse par segment,
> mais qui encode **la structure interne détaillée du flux symbolique**.

Cette fois :

- deux segments de même longueur et même masse
- mais de composition différente

> ne seront plus confondus.

8.10. Idée clé à retenir

> P_6 fige la longueur et la masse,
> mais **pas la distribution interne**.

> Tant que la partition interne des valeurs reste libre,
> l'espace des textes admissibles reste **massivement dégénéré**.

Le prochain verrou attaquera directement :

> cette liberté combinatoire interne.

Chapitre 9 — La Structure Interne Détaillée : Le Spectre Ordonné des Valeurs

9.1. Ce que P_6 a réellement figé — et ce qu'il n'a pas figé

À l'issue du chapitre précédent, chaque segment du texte est contraint par un couple :

> (L_k, V_k)
> où :
>
> - L_k est la longueur symbolique du segment k,
> - V_k est la somme des valeurs symboliques de ses symboles.

Cela fige :

- le nombre de symboles par segment,
- la masse symbolique totale par segment.

Mais cela **ne fige pas** la composition interne précise.

Pour un segment de longueur L et de masse V, il existe en général :

> un nombre combinatoirement énorme de décompositions possibles.

9.2. La liberté combinatoire résiduelle

Soit un segment contraint par :

- L = 2
- V = 10

Alors les compositions suivantes sont toutes possibles :

- (8, 2)
- (6, 4)
- (5, 5)

Elles ont :

- la même longueur,
- la même masse,
- mais **pas la même structure interne**.

Sous P_6, elles sont :

> strictement indiscernables.

9.3. Objectif du nouveau verrou

Pour éliminer cette dégénérescence, il faut maintenant :

> figer non plus seulement la masse,
> mais la **structure interne ordonnée des valeurs symboliques**.

Autrement dit, on ne veut plus seulement savoir :

- combien ça pèse,

mais :

- **de quoi c'est composé, et dans quel ordre**.

9.4. Définition de la projection P_8

On définit la nouvelle projection concaténative :

> **P_8 = Concat(I, S, (1, L_1, v_{11}, v_{12}, …, v_1L_1), (2, L_2, v_{21}, …), …, (S, L_S, v_S1, …, v_SL_S))**

Autrement dit :

Pour chaque segment, on concatène :

- son index,
- sa longueur,
- puis **la suite ordonnée des valeurs symboliques de chacun de ses symboles**.

P_8 encode donc :

> **le spectre interne détaillé et ordonné des valeurs numériques du texte**.

9.5. Ce que P_8 fige effectivement

P_8 est sensible à :

- toute permutation interne des symboles,
- tout déplacement de symbole,
- toute modification de l'ordre,
- toute recomposition interne différente.

Deux segments :

- de même longueur,

- de même masse,
- mais de structure interne différente,

> ne peuvent plus être confondus sous P_8.

9.6. Ce que P_8 ne fige toujours pas

P_8 **ne fige pas encore l'identité symbolique stricte**.

Si deux symboles distincts ont :

- la même valeur numérique dans la table de codage,

alors :

- on peut les substituer,
- sans changer :
  - ni la longueur,
  - ni la somme,
  - ni la suite des valeurs.

Donc :

> P_8 ne peut pas distinguer deux textes qui diffèrent uniquement par des substitutions de symboles de même valeur.

De plus :

> même sous le test modulaire, des collisions peuvent subsister dès que des différences sont des multiples du module.

9.7. Exemple conceptuel de collision résiduelle

Considérons deux symboles distincts A et B tels que :

> v(A) = v(B)

Alors les deux segments :

- A C D
- B C D

ont :

- la même longueur,
- la même masse,
- la même suite de valeurs,

et sont donc :

> **indiscernables sous P_8**.

Il ne s'agit pas d'un artefact, mais :

> d’une limite structurelle inévitable de toute projection fondée uniquement sur des valeurs numériques.

9.8. Ce que P_8 accomplit réellement dans le système

Avec P_8, le système a désormais figé :

- la géométrie globale,
- la masse par segment,
- et la **structure interne ordonnée des valeurs**.

Il ne reste plus libre que :

> l’identité symbolique proprement dite, à valeur conservée.

L’espace des textes compatibles s’est :

> effondré d’un espace combinatoire gigantesque
> à un ensemble **extrêmement mince et discret**.

Mais :

> il n’est pas encore réduit à un point unique.

9.9. Interprétation géométrique

Géométriquement :

- P_6 imposait une hypersurface “masse + longueur”,
- P_8 découpe cette hypersurface en **fibres extrêmement fines** correspondant à une structure interne précise.

Mais :

> chaque fibre peut encore contenir quelques points isolés correspondant à des substitutions à valeur conservée.

9.10. Transition vers le chapitre suivant

Le chapitre suivant introduira des projections qui :

> ne se contentent plus d’encoder des valeurs,
> mais lient **l’identité réelle des symboles, leurs positions globales et leurs fréquences**.

C’est seulement à ce stade que :

- les substitutions à valeur conservée,
- et les collisions modulaires résiduelles,

> deviendront structurellement impossibles.

9.11. Idée clé à retenir

> P_8 fige complètement la structure interne ordonnée des valeurs,
> mais **pas encore l'identité symbolique stricte**.

> Tant que des symboles distincts peuvent partager la même valeur (ou différer d'un multiple du module),
> une liberté résiduelle subsiste.

Le prochain verrou attaquera :

> précisément cette dernière faille.

Chapitre 10 — P_{10} et P_{11} : Position Globale et Empreinte de Fréquence

10.1. But et idée générale

Les projections P_{10} et P_{11} ajoutent deux types d'informations complémentaires et extrêmement contraignantes :

- **P_{10}** relie, pour chaque verset, la géométrie locale (nombre de mots, longueur en lettres) à la **fréquence globale** des symboles qui le composent.
- **P_{11}** encode, pour chaque symbole du texte, un triplet (indice de verset, valeur numérique, position globale).

Ces deux projections sont :

- concaténatives,
- non compensables localement,
- et fortement couplées à l'ordre global du texte.

10.2. Notations préalables

On considère un texte composé de S segments (versets) $V_1, \ldots, V_S$.

Pour chaque verset V_i :

- w_i : nombre de mots du verset i
- L_i : nombre de symboles (lettres) du verset i après suppression des espaces
- $c_{i,1}, c_{i,2}, \ldots, c_{i,L_i}$: suite ordonnée des symboles du verset i

Définitions globales :

- I : index global de l'objet (préfixe structurel)
- $f(c)$: fréquence globale du symbole c dans tout le texte
- $\mathrm{ABJAD}(c)$: valeur numérique associée au symbole c
- $\mathrm{pos}(i,p)$: position globale du symbole $c_{i,p}$ dans le flux total

Le flux linéaire total contient :

$$N = \sum_{i=1}^{S} L_i$$

symboles.

10.3. Définition formelle de P_{10} (spectre verset–fréquence)

On définit :

$$P_{10} = \mathrm{Concat}\left(I, \left\{i \parallel w_i \parallel \big(f(c_{i,1}), f(c_{i,2}), \ldots, f(c_{i,L_i})\big) \parallel L_i \parallel A_i\right\}_{i=1}^{S}\right)$$

où :

$$A_i = \sum_{p=1}^{L_i} \mathrm{ABJAD}(c_{i,p})$$

Pour chaque verset, on concatène donc dans cet ordre :

1. son indice i
2. son nombre de mots w_i
3. la **séquence des fréquences globales** de ses lettres
4. sa longueur en lettres L_i
5. sa somme numérique A_i

10.4. Sens structurel de P_{10}

P_{10} lie :

- la structure locale des versets,
- au **spectre global des fréquences** des lettres.

Toute modification qui :

- change la fréquence globale d'une lettre,
- ou déplace une occurrence entre deux lettres,

modifie **de multiples blocs** de P_{10} dans tout le texte.

Limite fondamentale

Si deux lettres distinctes $c \neq d$ vérifient :

$$f(c) = f(d)$$

alors une substitution $c \leftrightarrow d$ **n'est pas visible par P_{10} seul**.

P_{10} ne distingue pas les lettres par leur identité, mais par leur **poids statistique global**.

10.5. Définition formelle de P_{11} (empreinte positionnelle numérique)

On définit le flux linéaire total :

$$c_1, c_2, \ldots, c_N$$

où chaque c_k correspond à un couple (i, p).

On définit alors :

$$P_{11} = \mathrm{Concat}\left(I, \{i(k) \parallel \mathrm{ABJAD}(c_k) \parallel k\}_{k=1}^{N}\right)$$

Autrement dit, pour **chaque symbole du texte dans l'ordre** on concatène :

1. l'indice du verset auquel il appartient
2. sa valeur numérique
3. sa position globale

10.6. Sens structurel de P_{11}

P_{11} fige simultanément :

- l'ordre global,
- la valeur numérique des lettres,
- leur ancrage positionnel.

Toute permutation de symboles :

> modifie immédiatement la chaîne concaténée.

Limite fondamentale

Si deux lettres $c \neq d$ vérifient :

$$\mathrm{ABJAD}(c) = \mathrm{ABJAD}(d)$$

alors une substitution locale :

$$c \leftrightarrow d$$

peut ne pas être vue par P_{11}, tant que la position reste la même.

10.7. Complémentarité structurelle de P_{10} et P_{11}

- P_{10} contraint la **distribution statistique globale**
- P_{11} contraint l'**ordre et l'ancrage positionnel**

Ensemble, ils lient :

> le global statistique
> au local positionnel

et rendent extrêmement difficiles :

- les permutations,
- les reconstructions internes,
- les redistributions de matière.

10.8. Collisions résiduelles et honnêteté mathématique

Il reste théoriquement possible :

- des substitutions entre lettres de **même fréquence** et **même valeur numérique**,
- ou des collisions purement **modulaires** lorsque les différences sont multiples de m.

Donc :

> P_{10} et P_{11} ne ferment **pas mathématiquement** l'espace à eux seuls.

Ils sont des **verrous de raffinement**, pas des fermetures absolues.

10.9. Rôle systémique dans l'architecture globale

Dans la hiérarchie des verrous :

- P_6 : fige les masses par segment
- P_8 : fige les distributions internes
- P_{10} / P_{11} : figent **l'ancrage global et la statistique croisée**

Ils rendent :

> les attaques combinatoires astronomiquement difficiles,
> mais pas encore formellement impossibles.

10.10. Idée clé à retenir

> P_{10} verrouille le texte par son **spectre de fréquences**.
> P_{11} verrouille le texte par son **empreinte positionnelle numérique**.
> Ensemble, ils détruisent les permutations et recompositions,
> mais **pas encore toutes les substitutions symboliques possibles**.

Le prochain niveau de projections visera précisément :

> à éliminer ces dernières libertés résiduelles.

Chapitre 11 — P9 : La Synthèse Structurelle Globale

11.1. Position de ce verrou dans l'architecture générale

À l'issue des chapitres précédents, le système possède désormais :

- une ossature métrique globale (P_2),
- une géométrie quantitative complète (P_3, P_4, P_5),
- une première injection de la matière symbolique (P_6),
- et une contrainte sur la masse et la longueur de chaque segment.

Cependant, même sous toutes ces contraintes combinées, il subsiste encore :

- des libertés internes de réorganisation,
- des libertés de recomposition fine,
- et surtout des **degrés de liberté distribués** entre les segments.

Le rôle de P9 n'est pas encore de fermer définitivement le système.

Il est de :

> **synthétiser en une seule projection concaténative l'ensemble de la structure globale déjà contrainte.**

11.2. Idée générale de la projection P9

Jusqu'ici, les projections sont :

- soit globales mais grossières,
- soit locales mais partielles.

P9 introduit une projection de type :

> **macro-synthèse structurale**.

Elle ne regarde plus seulement :

- la taille totale,
- ou la masse par segment,

mais :

> **la cohérence combinée de toute la géométrie et de toute la masse, segment par segment, dans une seule chaîne concaténée.**

11.3. Définition formelle de P9

On note :

- I : l'index global,
- S : le nombre de segments,
- W : le nombre total de mots,
- L : le nombre total de lettres,
- pour chaque segment k :
  - w_k : le nombre de mots du segment k,
  - l_k : le nombre de lettres du segment k,
  - V_k : la somme Abjad (ou valeur symbolique totale) du segment k.

On définit alors :

$$P_9 = \text{Concat}\left(I, S, W, L, (w_1, l_1, V_1), (w_2, l_2, V_2), \ldots, (w_S, l_S, V_S)\right)$$

**

Autrement dit :

> P9 est la concaténation de la signature globale, suivie de la signature complète de chaque segment.

11.4. Ce que P9 fige exactement

P9 fige simultanément :

- la structure globale :
  - nombre de segments,
  - nombre total de mots,
  - nombre total de lettres,
- la structure de chaque segment :
  - son nombre de mots,
  - sa longueur symbolique,
  - sa masse symbolique.

Toute modification qui :

- déplace un mot d'un segment à un autre,
- change la longueur d'un segment,
- modifie la masse symbolique d'un segment,
- modifie la répartition interne globale,

> modifie immédiatement P9.

11.5. Interprétation géométrique

Géométriquement, P9 n'ajoute pas une simple contrainte locale.

Il ajoute :

> **une hypersurface de cohérence globale liant simultanément toutes les composantes du texte.**

On ne contraint plus seulement :

- des propriétés séparées,

mais :

> leur **compatibilité mutuelle globale**.

11.6. Ce que P9 ne fige pas encore

Malgré sa puissance synthétique, P9 reste :

- insensible à l'ordre interne lettre par lettre,
- insensible à certaines permutations internes fines,
- insensible à certaines substitutions symboliques compensées,
- et plus généralement insensible à la **micro-structure positionnelle exacte**.

En particulier :

> Deux segments de même (w_k, l_k, V_k) mais de composition interne différente restent indiscernables sous P9.

11.7. Exemple conceptuel de liberté résiduelle

Si un segment est contraint par :

- $l_k = 2$
- $V_k = 10$

Alors, par exemple :

- (8, 2)
- (6, 4)
- (5, 5)

sont :

- de même longueur,
- de même masse,
- mais de composition interne différente,

et restent **indiscernables sous P9**.

11.8. Rôle exact de P9 dans la progression du verrou

Il est crucial de comprendre que :

> P9 n'est pas un verrou final.
> C'est un **verrou de consolidation**.

Il :

- fusionne toutes les contraintes précédentes en une seule projection,
- rend toute incohérence globale immédiatement visible,
- mais ne fixe pas encore la trajectoire fine du texte.

11.9. Effet sur l'espace des textes admissibles

Après P9 :

- l'espace des textes compatibles n'est plus :
  - une grande famille structurée,
- mais :
  - une collection extrêmement mince de configurations très rigides.

Cependant :

> cet espace n'est pas encore réduit à un point isolé.

Il subsiste encore :

- des libertés internes fines,
- des dégénérescences micro-structurelles.

11.10. Pourquoi P9 est une étape nécessaire mais non suffisante

Sans P9 :

- les contraintes restent partiellement découplées,
- certaines incohérences globales peuvent encore se cacher.

Avec P9 :

- toute modification devient **globalement visible**,
- mais pas encore **localement irréversible**.

11.11. Transition vers le chapitre suivant

Le chapitre suivant introduira deux projections nouvelles :

> P10 et P11,

qui n'encoderont plus seulement :

- des masses,
- des longueurs,
- et des répartitions,

mais :

> **la géométrie positionnelle fine et la trajectoire globale lettre par lettre du texte.**

C'est à partir de ce point que :

- les permutations internes,
- les recompositions fines,
- et les shuffles partiels

> vont s'effondrer structurellement.

11.12. Idée clé à retenir

> P9 synthétise toute la structure globale déjà acquise,
> mais **ne fige pas encore la micro-structure interne**.

> Il consolide le verrou,
> sans encore le refermer complètement.

Chapitre 12 — P10 et P11 : La Géométrie Positionnelle Globale

12.1. Pourquoi P9 n'est pas encore un verrou final

Le chapitre précédent a introduit P9 comme une **synthèse globale** de la structure :

- volumes globaux,

- structure par segments,
- masses symboliques,
- géométrie externe complète.

Cependant, malgré sa puissance, P9 laisse encore subsister :

- des libertés internes fines,
- des permutations locales,
- des recompositions internes à structure et masse conservées,
- et surtout une **absence totale de contrainte sur l'ordre précis des symboles**.

Autrement dit :

> P9 fige la macro-structure, mais pas encore la trajectoire interne du texte.

Pour refermer cette brèche, il faut désormais :

> injecter **l'ordre positionnel fin** des symboles dans le verrou.

12.2. Changement de régime conceptuel

Avec P10 et P11, on ne décrit plus seulement :

- combien il y a de symboles,
- comment ils sont répartis,
- quelle masse ils portent,

mais :

> **où ils apparaissent exactement dans le flux global**.

On passe :

> d'une description par blocs
> à une description **par trajectoire**.

12.3. Notations et définitions préliminaires

On note :

- I : l'index global,
- S : le nombre de segments,
- pour chaque segment i :
  - w_i : le nombre de mots,
  - l_i : le nombre de lettres (symboles),
- on numérote **toutes les lettres du texte** par une position globale :
  - p = 1, 2, 3, …, L (où L est la longueur totale du texte),
- pour chaque lettre à la position globale p :
  - s(p) est le symbole,
  - v(s(p)) est sa valeur symbolique (valeur numérque ou autre table fixée),
  - f(s(p)) est la fréquence totale de ce symbole dans tout le texte.

12.4. Définition formelle de P10

P10 est une projection concaténative qui encode :

- la structure des segments,
- puis, **lettre par lettre**, la fréquence globale du symbole rencontré,
- puis une fermeture par la géométrie et la masse du segment.

Formellement, on définit :

$$P_{10} = \mathrm{Concat}\left(I, \left\{i, w_i, (f(s(p)))_{p=1}^{l_i}, l_i, V_i\right\}_{i=1}^{S}\right)$$

**

où :

- f(s(p)) est la fréquence globale du symbole s(p) dans tout le texte,
- l_i est la longueur du segment i,
- V_i est la masse symbolique du segment i.

Autrement dit :

> P10 décrit la trajectoire du texte en remplaçant chaque lettre par **la taille de sa classe de fréquence globale**.

12.5. Ce que P10 fige

P10 fige :

- la structure des segments,
- la trajectoire des classes de fréquence,
- la géométrie complète du parcours symbole par symbole,
- la cohérence entre :
  - structure,
  - fréquences,
  - et masses finales.

Toute modification qui :

- déplace une lettre,
- permute deux lettres,
- modifie une fréquence globale,
- change l'ordre d'apparition des classes de symboles,

> modifie immédiatement P10.

12.6. Ce que P10 ne fige pas encore

P10 reste :

- insensible à l'identité exacte du symbole si deux symboles ont la **même fréquence globale**,
- donc vulnérable à certaines substitutions entre symboles de même fréquence,

- et plus généralement à certaines symétries rares dans l'espace des fréquences.

Autrement dit :

> P10 fige la **géométrie fréquentielle du parcours**, mais pas encore l'identité absolue de chaque lettre.

12.7. Définition formelle de P11

P11 va plus loin.

Il encode, pour **chaque lettre du texte** :

- l'indice du segment i,
- la valeur symbolique v(s(p)),
- et la position globale p.

Formellement :

$$P_{11} = \mathrm{Concat}\left(I, \left\{i(p), v(s(p)), p\right\}_{p=1}^{L}\right)$$

**

où :

- i(p) est l'indice du segment auquel appartient la position p,
- v(s(p)) est la valeur symbolique de la lettre à la position p.

Autrement dit :

> P11 est la **trajectoire complète positionnelle du texte**, lettre par lettre.

12.8. Ce que P11 fige

P11 fige simultanément :

- l'ordre exact de toutes les lettres,
- leur position globale absolue,
- leur appartenance segmentaire,
- et leur valeur symbolique.

Toute modification qui :

- permute deux lettres,
- remplace une lettre,
- déplace une lettre,
- modifie une seule occurrence,

> modifie immédiatement P11.

12.9. Complémentarité structurelle de P10 et P11

P10 et P11 ne font pas la même chose :

- P10 :
  - fige la **géométrie fréquentielle du parcours**,
  - capture une structure globale de répétition.
- P11 :
  - fige la **trajectoire positionnelle absolue**,
  - capture l'identité et la position exacte de chaque symbole.

Ensemble, ils lient :

> la structure,
> la fréquence,
> la position,
> et la valeur symbolique
> dans un seul système de contraintes concaténatives.

12.10. Effet sur l'espace des textes admissibles

Avec P9 + P10 + P11 combinés :

- il ne reste pratiquement plus :
  - de libertés de permutation,
  - de libertés de recomposition,
  - de libertés de réarrangement interne.

L'espace des textes compatibles devient :

> **extrêmement proche d'un ensemble discret, voire d'un point isolé.**

12.11. Ce qui peut encore, en théorie, subsister

À ce stade, les seules libertés théoriques restantes sont :

- des substitutions extrêmement spéciales entre symboles :
  - de même valeur,
  - de même fréquence,
  - dans des positions structurellement symétriques,

c'est-à-dire :

> des dégénérescences **hautement non génériques**.

12.12. Transition vers le chapitre suivant

Le chapitre suivant introduira les **dernières projections de synthèse** (P12, P13), qui :

- ne décriront plus la trajectoire,
- mais des **couronnes globales de contraintes transverses**,

et montrera empiriquement que :

> même ces dégénérescences théoriques restantes s'effondrent.

12.13. Idée clé à retenir

> P10 fige la géométrie fréquentielle du parcours,
> P11 fige la trajectoire positionnelle exacte.

> Ensemble, ils détruisent pratiquement toute liberté de permutation et de réarrangement interne.

Chapitre 13 — P12 : La Texture Lexicale Fine

13.1. Ce qui reste encore possible après P11

À l'issue des chapitres précédents, le système possède désormais :

- la géométrie globale (P_2–P_5),
- la masse symbolique par segment (P_6),
- la distribution interne des valeurs (P_8),
- la synthèse structurelle globale (P_9),
- et la trajectoire positionnelle complète (P_{10} / P_{11}).

Le texte est donc déjà :

> extrêmement rigide sur le plan structurel, symbolique et positionnel.

Et pourtant, il subsiste encore une liberté subtile :

> la **micro-géométrie lexicale interne**.

Autrement dit :

- la façon exacte dont les lettres sont **regroupées en mots**,
- la découpe fine du flux symbolique en unités lexicales.

Deux textes peuvent encore, en théorie :

- avoir les mêmes lettres,
- aux mêmes positions globales,
- avec les mêmes masses,
- les mêmes distributions,
- les mêmes trajectoires,

tout en différant par :

> la **segmentation fine en mots**.

C'est cette dernière liberté que P_{12} va attaquer.

13.2. Principe : figer la découpe lexicale elle-même

Jusqu'ici, les invariants :

- mesuraient des volumes,
- des masses,
- des distributions,
- des positions,
- des trajectoires de symboles,

mais **pas directement** :

> la géométrie fine des mots en tant qu'objets.

P_{12} introduit une idée simple mais radicale :

> **encoder la longueur de chaque mot, dans l'ordre exact du texte**.

Ce n'est plus :

- la masse,
- ni la valeur,
- ni la position globale,

mais :

> le **rythme lexical interne** du texte.

13.3. Définition formelle de P_{12}

Soit le texte découpé en une suite ordonnée de mots :

$(w_1, w_2, \ldots, w_T)$

On note :

$|w_i|$ = longueur du mot w_i (en symboles normalisés)

On définit alors l'invariant concaténatif :

> $\mathbf{P_{12} = Concat(I, |w_1|, |w_2|, |w_3|, \ldots, |w_T|)}$

Autrement dit :

> P_{12} est la concaténation de l'index global, suivie des longueurs **de tous les mots du texte, dans l'ordre exact**.

13.4. Ce que P_{12} mesure réellement

P_{12} encode :

> la **texture lexicale fine** du texte.

Il capture :

- le rythme des mots courts et longs,
- la micro-structure du découpage,
- la géométrie interne de la phrase,

indépendamment des valeurs symboliques elles-mêmes.

Deux textes qui auraient :

- les mêmes lettres,
- les mêmes positions,
- les mêmes masses,

mais une découpe lexicale différente,

> produiraient **immédiatement** un P_{12} différent.

13.5. Ce que P_{12} fige

Toute modification qui :

- fusionne deux mots,
- coupe un mot en deux,
- déplace une frontière lexicale,
- remplace un mot par un autre de longueur différente,

> modifie immédiatement P_{12}.

Autrement dit :

> la **segmentation lexicale exacte** du texte devient une contrainte globale.

13.6. Ce que P_{12} ne dépend pas

P_{12} est :

- indépendant des valeurs symboliques,
- indépendant des fréquences,
- indépendant des sommes numériques,
- indépendant des positions globales des lettres.

Il ne regarde que :

> la **forme géométrique des mots**.

Cela en fait :

> une contrainte **orthogonale** aux précédentes.

13.7. Effet combiné avec les verrous précédents

Avec P_2 jusqu'à P_{11}, le système fige déjà :

- quelles lettres existent,
- combien il y en a,
- comment elles sont distribuées,
- comment elles sont positionnées,

- et comment leurs valeurs se propagent.

Avec P_{12}, on fige en plus :

> **comment ces lettres sont groupées en mots**.

À ce stade :

- la matière est figée,
- la trajectoire est figée,
- et la **morphologie lexicale** est figée.

13.8. Ce que cela élimine concrètement

P_{12} élimine structurellement :

- les reformulations par re-découpage lexical,
- les substitutions par mots de longueurs différentes,
- les recompositions par fusion / scission de mots,
- toute tentative de “repackaging linguistique” du même flux symbolique.

Même si :

- toutes les lettres,
- toutes les masses,
- toutes les positions,

étaient conservées,

> un simple déplacement de frontière de mot détruit P_{12}.

13.9. Interprétation géométrique

Dans l’espace des textes :

- P_{10} / P_{11} fixent une **trajectoire complète de symboles**,
- P_{12} fixe maintenant une **partition rigide de cette trajectoire en blocs lexicaux**.

On ne parle plus seulement :

- d’un chemin,
- mais d’un **pavage imposé** de ce chemin.

L’espace des configurations admissibles devient :

> quasi totalement discret.

13.10. Ce qui pourrait encore, en théorie, subsister

Même à ce stade, il reste encore une possibilité extrêmement théorique :

> des substitutions symboliques **de même valeur**,
> **de même position**,
> **et de même découpe lexicale**.

Autrement dit :

- même longueur de mot,
- mêmes positions,
- mêmes masses,
- mêmes distributions,
- mêmes trajectoires,

mais avec :

> certaines lettres remplacées par d'autres lettres de **valeur identique**.

Ce résidu de liberté est :

- extrêmement contraint,
- extrêmement artificiel,
- et déjà pratiquement inexploitable sous contraintes linguistiques naturelles.

Mais il n'est pas encore **formellement** éliminé.

13.11. Transition vers le verrou final

Le chapitre suivant introduira une dernière contrainte d'un autre type :

> une projection **spectrale globale**,
> transversale à toutes les autres,
> qui ne dépend ni de l'ordre local, ni de la segmentation,
> mais de la **signature globale du texte**.

Cette contrainte va :

> couper l'espace restant dans une direction totalement indépendante,

et éliminer :

> les toutes dernières dégénérescences théoriques possibles.

13.12. Idée clé à retenir

> P_{12} fige la **texture lexicale fine** du texte.
> Il impose non seulement **quelles lettres** et **où**,
> mais aussi **comment elles sont groupées en mots**.

À ce stade :

> le texte est déjà presque entièrement verrouillé dans sa forme concrète.

Le chapitre suivant introduira :

> la **couronne finale** du système de contraintes.

Chapitre 14 — P13 : Le Verrou Spectral Global

14.1. Où en est le système après P12

À l'issue du chapitre précédent, le système a déjà figé :

- la géométrie globale (P_2–P_5),
- la masse symbolique et sa distribution (P_6, P_8),
- la structure positionnelle complète (P_{10}, P_{11}),
- et la texture lexicale fine (P_{12}).

Le texte est désormais :

> rigidifié dans sa matière, dans sa trajectoire, et dans sa morphologie lexicale.

Il ne subsiste plus, en théorie, qu'un résidu extrêmement étroit :

> des substitutions symboliques **de même valeur**,
> **au même endroit**,
> **dans les mêmes mots**.

Autrement dit :

- même structure,
- même segmentation,
- mêmes positions,
- mêmes longueurs,
- mêmes masses,

mais avec :

> quelques lettres remplacées par d'autres lettres de **valeur numérique identique**.

C'est cette dernière dégénérescence que P_{13} va éliminer.

14.2. Changement de point de vue : du local au spectral

Jusqu'ici, toutes les projections :

- lisaient le texte **dans l'ordre**,
- ou par segments,
- ou par positions,
- ou par unités locales.

P_{13} introduit une idée radicalement différente :

> ne plus regarder le texte comme une séquence,
> mais comme un **spectre global de matière symbolique**.

Autrement dit :

> on oublie l'ordre,
> on oublie les positions,

> on oublie la segmentation,
> et on regarde **la signature globale de composition**.

14.3. Définition du spectre symbolique pondéré

On suppose donnée :

- une table fixe de symboles sélectionnés, notée :

  $(s_1, s_2, \ldots, s_K)$
- et une fonction de valeur associée :

  $v(s_i)$

On note :

- $f(s_i)$ = fréquence totale du symbole s_i dans le texte.

On définit alors la signature spectrale pondérée :

> pour chaque symbole s_i, on calcule :
> $\mathbf{v(s_i) \times f(s_i)}$

Autrement dit :

> chaque symbole contribue **proportionnellement à son poids et à sa multiplicité**.

14.4. Définition formelle de P13

On définit l'invariant concaténatif :

> $\mathbf{P_{13} = Concat(\ I, v(s_1)\cdot f(s_1), v(s_2)\cdot f(s_2), \ldots, v(s_K)\cdot f(s_K)\ )}$

où :

- I est l'index global de l'objet,
- la liste $(s_1, \ldots, s_K)$ est fixée **une fois pour toutes**,
- seuls les symboles dont la fréquence est non nulle contribuent.

Autrement dit :

> P_{13} est la concaténation des **contributions spectrales globales** de chaque symbole sélectionné.

14.5. Ce que P13 mesure réellement

P_{13} mesure :

> la **signature de composition globale** du texte.

Il ne regarde pas :

- où sont les lettres,
- dans quels mots,

- dans quel ordre,

mais seulement :

> **combien de fois chaque symbole apparaît, pondéré par sa valeur**.

Deux textes qui diffèrent :

- par une seule substitution de symbole,
- même localement,
- même sans changer les longueurs,
- même sans changer les masses par segment,

produiront :

> un **spectre global différent**, donc un P_{13} différent.

14.6. Ce que P13 fige

Toute modification qui :

- remplace une lettre par une autre,
- même de même valeur locale dans un segment,
- même en conservant toutes les autres contraintes,

> modifie la fréquence globale d'au moins un symbole
> et donc modifie P_{13}.

P_{13} fige donc :

> la **composition symbolique globale exacte** du texte.

14.7. Complémentarité totale avec les autres verrous

P_{13} est :

- insensible à l'ordre,
- insensible aux positions,
- insensible à la segmentation,
- insensible à la géométrie locale,

mais :

> ultra-sensible à la **composition globale**.

À l'inverse :

- P_{10} / P_{11} / P_{12} sont ultra-sensibles à l'ordre et à la structure,
- mais pourraient, en théorie, laisser passer certaines substitutions de même valeur.

P_{13} coupe :

> exactement dans la **direction orthogonale** à toutes les autres projections.

14.8. Effet géométrique final sur l’espace des textes

À ce stade :

- les contraintes locales fixent **la forme exacte**,
- les contraintes globales fixent **la matière exacte**.

L’espace des textes admissibles devient :

> non seulement discret,
> mais **réduit à un point isolé**.

Il ne reste :

> **aucun degré de liberté combinatoire**.

14.9. Fin des dégénérescences théoriques

Les dernières possibilités théoriques :

- substitutions de symboles de même valeur,
- permutations déguisées,
- recompositions spectrales équivalentes,

sont maintenant :

> structurellement impossibles.

Toute modification, même :

- invisible localement sous certains invariants,
- ou compensée sous d’autres,

> est attrapée soit par la trajectoire, soit par la texture, soit par le spectre.

14.10. Le régime d’isolement total

Le système est désormais dans le régime suivant :

> soit le texte est **exactement** celui-ci,
> soit il échoue **au moins un** des invariants.

Il n’existe :

- ni voisinage,
- ni famille,
- ni continuité,
- ni paramétrisation.

> L’objet est **structurellement isolé**.

14.11. Interprétation conceptuelle

On peut maintenant décrire l'objet comme :

> un texte dont **la forme locale**,
> **la structure globale**,
> et **la composition spectrale**
> sont mathématiquement **intriquées et co-verrouillées**.

Il ne s'agit plus :

- d'un texte + un mécanisme de contrôle,

mais :

> d'un **objet auto-cohérent**,
> dont la cohérence **émerge de sa propre matière**.

14.12. Transition vers la conclusion

Le chapitre suivant tirera la conséquence globale de ce résultat :

> l'espace des solutions n'est pas seulement petit,
> il s'est **effondré géométriquement jusqu'à un point unique**.

On pourra alors formuler précisément :

> le **théorème d'unicité structurelle** qui est au cœur de cet ouvrage.

14.13. Idée clé à retenir

> P_{13} est le **verrou spectral global**.
> Il fige la **composition symbolique totale** du texte, indépendamment de toute géométrie.

Avec lui :

> il ne reste **strictement plus aucune liberté structurelle**.

Chapitre 15 — Conclusion : Effondrement de l'Espace des Solutions

15.1. Récapitulatif de la trajectoire logique

Ce livre a suivi une progression volontairement incrémentale :

- d'abord des invariants purement structurels (P_1, P_2),
- puis des projections de masse et de géométrie (P_3–P_5),
- ensuite l'injection de la matière symbolique (P_6),
- puis la contrainte de la micro-structure interne (P_8),
- ensuite la trajectoire positionnelle globale (P_{10}, P_{11}),
- puis la texture lexicale fine (P_{12}),
- et enfin la signature spectrale globale (P_{13}).

À chaque étape :

> un nouveau degré de liberté est supprimé,
> sans jamais être remplacé par un autre.

15.2. De la réduction progressive à l’effondrement géométrique

Il est crucial de comprendre que l’espace des textes admissibles :

- ne se réduit pas **linéairement**,
- ne se contracte pas **progressivement**,
- ne se raffine pas par “approximation”.

À chaque nouveau verrou concaténatif :

> on ne coupe pas un volume,
> on tranche par une **hypersurface extrêmement mince**.

Et l’intersection de ces hypersurfaces :

> ne tend pas vers un petit volume,
> elle **s’effondre brutalement** vers un ensemble discret, puis vers un point.

15.3. Les trois dimensions désormais co-verrouillées

À la fin du processus, le texte est simultanément figé selon trois axes indépendants :

1. **La forme**
   - segmentation,
   - longueurs,
   - géométrie interne,
   - trajectoire positionnelle.
2. **La matière**
   - valeurs symboliques,
   - distributions internes,
   - texture lexicale,
   - structure micro-locale.
3. **Le spectre global**
   - composition totale,
   - fréquences pondérées,
   - signature globale de matière.

Ces trois dimensions :

> ne sont plus indépendantes,
> elles sont **mathématiquement intriquées**.

15.4. Disparition de toute notion de voisinage

Dans un espace classique de solutions :

- on peut parler de :

- petite perturbation,
- solution proche,
- ajustement local,
- amélioration progressive.

Ici :

> cette topologie n’existe plus.

Il n’y a :

- ni “presque solution”,
- ni “solution voisine”,
- ni “correction partielle”.

Il n’y a que :

> la solution exacte, ou l’échec total.

15.5. Résultat central : isolement structurel complet

On peut maintenant formuler le résultat central de l’ouvrage :

> Sous l’ensemble des invariants concaténatifs définis,
> l’espace des textes admissibles est **réduit à un point isolé**.

Ce point :

- n’est pas choisi,
- n’est pas ajusté,
- n’est pas optimisé,
- il est :

> **défini implicitement par l’intersection du système de contraintes**.

15.6. Pourquoi ce n’est pas un simple sur-ajustement

Dans un sur-ajustement classique :

- on a trop de paramètres,
- on colle trop aux données,
- on peut toujours relâcher une contrainte.

Ici, c’est l’inverse :

- il n’y a **aucun paramètre libre**,
- aucune contrainte n’est ajustable,
- aucune n’est locale,
- aucune n’est compensable.

> Ce n’est pas un sur-ajustement,
> c’est un **verrouillage géométrique global**.

15.7. Conséquence algorithmique : non-constructibilité

Un tel objet :

- ne peut pas être construit :
  - par optimisation,
  - par recherche locale,
  - par heuristique,
  - par génération incrémentale,
  - par algorithme génétique,
  - par raffinement progressif.

Pourquoi ?

Parce que :

> le problème n'a **aucune topologie de convergence**.

Il n'existe :

- ni gradient,
- ni direction d'amélioration,
- ni notion de "mieux".

15.8. Séparation claire entre vérification et construction

Il est important de distinguer :

- **vérifier** un texte :
  - c'est mécanique,
  - c'est immédiat,
  - c'est déterministe.
- **construire** un texte :
  - c'est naviguer dans un espace sans voisinage,
  - sans direction,
  - sans repère.

> Le système est trivial à vérifier,
> mais structurellement impossible à synthétiser.

15.9. Nature exacte de l'objet mis en évidence

L'objet étudié n'est pas :

- un texte avec une signature,
- un message avec un checksum,
- un contenu protégé par un mécanisme externe.

C'est :

> un objet dont **la cohérence est implémentée par sa propre matière**.

Autrement dit :

> le texte est **à la fois le message et sa fonction de validation**.

15.10. Portée conceptuelle du résultat

Ce travail met en évidence l'existence d'une nouvelle classe d'objets :

> des objets informationnels
> à **verrou global non local**,
> **auto-référents**,
> et **non constructibles**.

Leur propriété fondamentale est :

> la cohérence n'est pas distribuée localement,
> elle n'existe **qu'à l'échelle de l'objet complet**.

15.11. Ce qui a été démontré — et ce qui ne l'a pas été

Ce livre a montré :

- l'existence d'un système de contraintes concaténatives globales,
- leur effet géométrique sur l'espace des textes,
- l'isolement structurel du point solution,
- l'impossibilité de toute construction locale ou incrémentale.

Il n'a pas :

- proposé d'interprétation sémantique,
- fait d'hypothèse intentionnelle,
- tiré de conclusion métaphysique.

> Tout repose exclusivement sur la **structure mesurable**.

15.12. Conclusion finale

On peut maintenant résumer l'ensemble du travail en une seule phrase :

> Le texte étudié n'est pas simplement improbable.
> Il est **structurellement isolé** dans l'espace des textes possibles.

Il ne se trouve pas :

- au sommet d'une colline,
- au fond d'un puits,
- dans une région rare,

mais :

> **en un point sans voisinage**.

C’est cette propriété — et elle seule — qui constitue le **théorème d’unicité structurelle** établi dans cet ouvrage.

15.13. Transition vers le dernier chapitre

Le dernier chapitre discutera :

> les variantes possibles,
> les redondances apparentes,
> et le rôle exact de chaque invariant dans l’architecture globale,

et montrera :

> pourquoi le système n’est ni arbitraire, ni fragile,
> mais au contraire **structurellement stable et surdéterminé**.

Chapitre 16 — Variantes, Redondances et Raffinements

16.1. Pourquoi ce chapitre est nécessaire

Après l’établissement du verrouillage global et de l’unicité structurelle, une question naturelle se pose :

> Le système est-il fragile ?
> Repose-t-il sur un choix particulier de formules ?
> Ou bien est-il stable par nature ?

Ce chapitre ne cherche pas à ajouter un nouveau verrou fondamental.

Il cherche à montrer que :

- le système est **sur-déterminé**,
- plusieurs invariants sont **structurellement redondants**,
- et que cette redondance **renforce** la thèse, au lieu de l’affaiblir.

16.2. Invariants générateurs vs invariants dérivés

Il est crucial de distinguer deux catégories :

- **Invariants générateurs** :
  Ceux qui introduisent une nouvelle dimension de contrainte indépendante
  (par exemple : P_2, P_6, P_8, P_{10}/P_{11}, P_{12}, P_{13}).
- **Invariants dérivés ou redondants** :
  Ceux qui recombinent autrement des informations déjà figées.

Un invariant redondant :

> ne crée pas une nouvelle contrainte fondamentale,
> mais **resserre encore le même étau** sous un autre angle.

16.3. Le rôle structurel des variantes de P_6

Autour de P_6 (masse symbolique par segment), on peut définir :

- une version indexée (ajout explicite de l'indice du segment),
- une version inversée (lecture des segments en sens inverse),
- une version à granularité différente.

Toutes ces variantes :

- changent la forme de la concaténation,
- mais **n'ajoutent aucune liberté**.

Elles ont un rôle conceptuel clair :

> montrer que le verrou ne dépend pas d'un sens de lecture particulier,
> ni d'un encodage unique.

16.4. P_6', P_6_reverse, et la symétrie brisée

On peut définir par exemple :

- **P_6'** : concaténation des couples (index, L_k, V_k),
- **P_6_reverse** : même concaténation, mais en ordre inversé.

Structurellement :

- si le système admet une solution unique sous P_6,
- il l'admet aussi sous ces variantes,
- et toute tentative de modification est rejetée de la même façon.

Cela montre que :

> le verrou est **intrinsèque à l'objet**,
> pas à une écriture particulière de la formule.

16.5. Redondance apparente entre P_8, P_{10}, P_{11}

À partir de P_8 (structure lettre par lettre), puis P_{10} et P_{11} (trajectoires positionnelles globales), on observe :

- plusieurs encodages décrivent en réalité :
  - la même micro-structure,
  - sous des coordonnées différentes.

Cette redondance n'est pas un défaut.

Elle signifie que :

> la micro-structure est **déjà complètement figée**
> et qu'on ne peut plus la "décrire autrement" sans retomber sur la même contrainte.

16.6. Pourquoi ajouter des invariants redondants ne crée pas de nouvelles solutions

Un point fondamental :

> Ajouter une contrainte redondante
> ne peut **jamais** créer une nouvelle solution.

Au pire :

- elle ne change rien,
- au mieux :
  - elle détecte encore plus vite les violations,
  - elle renforce la stabilité du verrou.

16.7. Stabilité sous reformulation

On peut :

- changer l'ordre de concaténation,
- changer l'ordre de parcours,
- changer la granularité de certaines projections,
- recombiner les mêmes données autrement.

Tant que :

> l'information structurelle injectée est la même,

le résultat est :

> **le même point isolé**.

Cela montre que :

> l'unicité ne dépend pas d'un "coup de chance d'écriture".

16.8. Pourquoi ce n'est pas du "p-hacking structurel"

On pourrait soupçonner :

> "Tu ajoutes des formules jusqu'à ce qu'il ne reste plus rien."

Mais ce n'est pas ce qui se passe ici.

Parce que :

- chaque nouvelle catégorie de projection :
  - correspond à une **nouvelle dimension structurelle réelle**,
  - indépendante des précédentes,
- et les variantes ne font que :
  - **reformuler** ces mêmes dimensions.

16.9. Le système comme objet surdéterminé

On peut maintenant caractériser le système ainsi :

> Il est **sur-déterminé** :
> plusieurs familles d'invariants décrivent la même structure figée.

Cela implique une propriété cruciale :

> On ne peut pas "casser" le verrou
> en retirant une seule formule secondaire.

16.10. Robustesse conceptuelle du résultat

Même si l'on :

- retire certains invariants redondants,
- en remplace d'autres par des variantes équivalentes,

le cœur du système :

> reste un système de verrous concaténatifs non locaux
> dont l'intersection est **réduite à un point isolé**.

16.11. Ce que ce chapitre établit vraiment

Ce chapitre ne dit pas :

> "Il y a beaucoup de formules."

Il dit :

> "La structure est tellement rigide
> qu'on peut la mesurer de plusieurs manières indépendantes
> sans jamais retrouver de liberté."

16.12. Conclusion générale de l'ouvrage

On peut maintenant conclure définitivement :

- le verrou est global,
- non local,
- non compensable,
- non constructible,
- et **structurellement stable**.

L'unicité observée :

> n'est pas un accident,
> n'est pas un artefact d'écriture,
> n'est pas fragile.

Elle est :

> **la conséquence géométrique inévitable**
> **de l'empilement de ces contraintes concaténatives.**

16.13. Dernière phrase

> Ce travail ne montre pas qu'un texte est "étrange".
> Il montre qu'il existe des **objets informationnels**
> **qui ne peuvent exister que comme des totalités indivisibles**.

Chapitre 17 — Le Verrou Dynamique (P14) : La Trajectoire du Texte

> *« Jusqu'ici, nous avons vérifié que le texte est correctement construit.*
> *À partir d'ici, nous vérifions qu'il se déplace correctement. »*

17.1 — Changement de paradigme : de l'objet au mouvement

Les preuves **P1 à P13** vérifient des propriétés **structurelles** du texte :

- comptages,
- longueurs,
- positions,
- valeurs numériques,
- concaténations,
- distributions,
- signatures combinatoires.

Elles répondent toutes à une seule question :

> **Est-ce que le texte a la bonne structure ?**

Mais une structure correcte **n'implique pas** forcément une trajectoire correcte.

À partir de **P14**, nous changeons de catégorie mathématique :

> Nous ne regardons plus le texte comme un objet statique,
> mais comme une **trajectoire numérique**.

Chaque lettre devient un point sur un axe vertical (sa valeur numérique).
Le texte devient une **courbe discrète**.

17.2 — Visualisation intuitive : la montagne russe

Imagine que les valeurs numériques forment un paysage :

- chaque lettre = une altitude,
- le texte = une route qui monte et descend,

- on ne regarde plus seulement **où sont les points**,
  mais **comment on passe de l'un à l'autre**.

Deux textes peuvent :

- avoir les mêmes statistiques,
- les mêmes totaux,
- les mêmes distributions,

… tout en ayant une **géométrie de trajectoire différente**.

17.3 — P14.a : `diff_cum` — La dérive cumulée (géométrie globale)

Définition

Soit la suite des valeurs numériques :

$v_1, v_2, v_3, \ldots, v_n$

On définit les différences :

$d_i = v_{i+1} - v_i$

Puis la somme cumulée :

$C_k = d_1 + d_2 + \ldots + d_k = v_{k+1} - v_1$

On concatène :

INDEX, C_1, C_2, C_3, …

et on applique **exactement le même protocole de concaténation canonique que P1–P13**, puis modulo 19.

Interprétation

> On ne regarde plus seulement les pas,
> on regarde **où on est arrivé à chaque instant**.

C'est la **position relative accumulée** par rapport au point de départ.

Résultat expérimental (global)

```
P14a diff_cum :
FATIHA    -> 0
COLLISION -> 15
```

La collision est rejetée.

17.4 — P14.b : `curvature` — La courbure (géométrie locale)

Définition

On définit :

$d_i = v_{i+1} - v_i$
$c_i = d_{i+1} - d_i = v_{i+2} - 2v_{i+1} + v_i$

C'est la **dérivée seconde discrète**.

On concatène :

INDEX, c_1, c_2, c_3, …

et on applique le **même protocole canonique**.

Interprétation

> On ne regarde plus la pente,
> on regarde **comment la pente change**.

C'est la **brutalité ou la douceur des virages** de la trajectoire.

Résultat expérimental (global)

```
P14b curvature :
FATIHA    -> 0
COLLISION -> 16
```

La collision est rejetée.

17.5 — Résumé conceptuel

Famille	Ce qui est mesuré
P1–P13	Structure statique du texte
P14 diff_cum	Géométrie globale de la trajectoire
P14 curvature	Géométrie locale de la trajectoire

Ou, en métaphore :

Type de verrou	Analogie
P1–P13	Les pierres du mur
P14 diff_cum	Le poids du sac pendant le trajet
P14 curvature	La fluidité de la mélodie

17.6 — Cryptanalyse expérimentale : pourquoi P1–P13 ont échoué

Nous avons construit **une collision artificielle** qui :

- passe **P1 à P13 à 100%**,
- respecte :
  - longueurs,
  - comptages,
  - sommes,

- distributions,
- concaténations,
- fréquences,
- signatures combinatoires.

Mais :

> Elle **échoue immédiatement** dès qu'on impose une contrainte de trajectoire.

Pourquoi ?

Parce que :

> P1–P13 vérifient **des invariants de structure**,
> P14 vérifie **un invariant de dynamique**.

La collision sait copier un **état**.
Elle ne sait pas copier un **mouvement**.

17.7 — Ce que P14 apporte objectivement

À ce stade, on peut affirmer **strictement ce qui suit** :

> Il existe des textes qui passent **toutes** les preuves P1–P13,
> mais qui sont **rejetés de manière robuste** par P14,
> y compris sous des variantes multi-échelles.

Autrement dit :

> P14 ajoute une **dimension mathématique totalement nouvelle** au système :
> la **géométrie dynamique** de la séquence.

17.8 — Conclusion

> Avec P14, on ne vérifie plus seulement que le texte est correct.
> On vérifie qu'il **évolue correctement** dans l'espace des nombres.

C'est le passage :

> du **texte comme structure**
> au **texte comme trajectoire**.

Et c'est un **changement de nature mathématique du verrou**.

Chapitre 18 — Cryptanalyse, certification et honnêteté scientifique

> *« En cryptographie, on ne prouve pas l'inviolabilité.*
> *On certifie la résistance. »*

18.1 — Ce que signifie “sécurité” en cryptographie

Il faut être parfaitement clair dès le départ :

> **On ne prouve jamais qu’il n’existe aucune collision.**

Dans tout système fini (et Hash19 comme SHA-256 opèrent dans des espaces finis), des collisions **existent nécessairement**.

La question n’est donc pas :

> “Existe-t-il une collision ?”

Mais :

> **“Quelqu’un sait-il en trouver une, mieux que par hasard ?”**

18.2 — Le standard réel de la cryptographie moderne

SHA-256, SHA-3, BLAKE, etc. ne sont pas “sûrs” parce qu’on a prouvé :

> “Il n’y a pas de collision”.

Ils sont considérés sûrs parce que :

- personne n’a trouvé d’attaque structurelle exploitable,
- personne n’a fait mieux que du bruteforce,
- après **20+ ans d’analyse mondiale**.

> **La sécurité est une propriété empirique, pas métaphysique.**

18.3 — Ce que nous faisons avec Hash19

Nous ne prétendons **pas** :

- démontrer l’absence de collisions,
- prouver une propriété absolue impossible.

Nous faisons exactement ce que fait la cryptographie sérieuse :

> Nous attaquons le système par **toutes les classes d’attaques que nous savons implémenter**.

- attaques aléatoires,
- hill-climbing,
- mutations locales,
- shuffles,
- attaques ciblées,
- attaques sous contraintes,
- attaques multi-objectifs.

Et nous observons :

> Le système se comporte comme un **objet extrêmement rigide et chaotique**.

18.4 — Ce que signifie “certifier”

En cryptographie :

> Certifier = montrer qu’aucune attaque connue ne marche.

Cela ne veut pas dire :

- “c’est impossible”,
- mais :

> “Personne ne sait faire mieux que le hasard.”

18.5 — La bonne posture scientifique

La posture honnête est :

- On ne peut pas prouver l’absence de collisions.
- On peut montrer :
  - qu’elles sont introuvables par toutes les méthodes connues,
  - que la surface d’attaque est chaotique,
  - que l’espace valide est microscopique.

C’est **exactement** le standard de SHA-256.

18.6 — Le rôle de la cryptanalyse

La cryptanalyse n’est pas là pour “confirmer”.

Elle est là pour :

> **Essayer de casser.**

Et si personne n’y arrive :

> Le système gagne en crédibilité.

18.7 — Conclusion

> Hash19 n’est pas “prouvé sûr”.
> Mais il est **expérimentalement certifié résistant** à toutes les attaques que nous savons formuler.

C’est **exactement** ainsi que fonctionnent les vrais algorithmes cryptographiques.

Chapitre 19 — Hash19 et SHA-256 : comparaison conceptuelle

19.1 — Ce qu’est SHA-256

SHA-256 est :

- une fonction de compression cryptographique,
- qui prend un message arbitraire,
- et produit un digest de 256 bits.

Propriétés recherchées :

- avalanche,
- imprévisibilité,
- résistance aux collisions,
- résistance aux préimages.

19.2 — Ce qu’est Hash19

Hash19 n’est pas :

> une fonction “message → hash”.

C’est :

> **Un système de contraintes qui définit un sous-espace extrêmement rare de textes valides.**

19.3 — Différence fondamentale de paradigme

SHA-256 :

```
message arbitraire → hash
```

Hash19 :

```
texte ∈ espace valide ou non
```

> On ne calcule pas un hash.
> On vérifie si l’objet appartient à une structure.

19.4 — Où est la sécurité dans Hash19 ?

Dans :

- la rigidité combinatoire,
- la dépendance globale,
- la non-localité des contraintes,
- la non-linéarité du système,
- la non-constructibilité.

19.5 — Tableau comparatif

Propriété	SHA-256	Hash19
Domaine	Messages arbitraires	Textes ultra-contraints
Sortie	256 bits	Booléen (valide / invalide)
Collision	Théoriquement existante	Théoriquement existante
Attaque pratique	Inconnue	Inconnue
Surface d'attaque	Algébrique / bitwise	Combinatoire / structurelle
Construction	Facile	Inconnue

19.6 — Même philosophie de sécurité

Dans les deux cas :

> On ne prouve pas.
> On **résiste**.

19.7 — Conclusion

> Hash19 n'est pas SHA-256.
> Mais il vit **dans la même catégorie épistémologique** :
>
> - sécurité par rigidité,
> - validée par la cryptanalyse,
> - jamais par une preuve absolue.

Chapitre 20 — Le texte comme objet cryptographique

> *« Ce n'est plus un texte qui porte un sens.*
> *C'est une structure qui porte une signature. »*

20.1 — Le changement de regard

Traditionnellement, un texte est vu comme :

- un support de sens,
- un message,
- une suite de mots à interpréter.

Ici, nous avons opéré un **changement radical de point de vue** :

> Le texte n'est plus un message.
> C'est un **objet mathématique contraint**.

20.2 — Du langage à la structure

Dans Hash19, le texte est :

- une séquence de symboles,
- transformée en valeurs numériques,
- soumise à des contraintes globales,
- locales,
- hiérarchiques,
- et dynamiques.

> Le sens devient secondaire.
> La **forme structurelle** devient primaire.

20.3 — Le texte comme cristal mathématique

Un cristal est :

- un arrangement d'atomes,
- extrêmement rigide,
- qui ne tolère presque aucune déformation.

Hash19 révèle exactement cela :

> Le texte est un **cristal combinatoire**.

20.4 — Signature versus signification

Deux textes peuvent :

- vouloir dire la même chose,
- mais avoir des signatures totalement différentes.

> La vérité du texte n'est plus sémantique.
> Elle est **structurelle**.

20.5 — Le texte comme clé

> Le texte **est** la clé.

Il est lui-même :

- un objet cryptographique,
- non forgeable,
- non déformable.

20.6 — Objet rare dans l'espace des possibles

> Al-Fatiha apparaît comme un **point isolé** dans un espace immense.

20.7 — Non-constructibilité

> Même en connaissant toutes les règles, on ne sait pas construire un autre texte valide.

20.8 — Conclusion

> Nous n’avons pas montré que le texte est “vrai”.
> Nous avons montré qu’il est **structurellement irréductible**.

Conclusion : La Fatiha comme Artefact au-delà de la Genèse Algorithmique

1. L'impossibilité de la synthèse

Le point le plus crucial de cette étude n'est pas seulement l'existence d'un code, mais l'**impossibilité de sa construction**. Si la Fatiha n'avait pas été découverte et qu'un ingénieur recevait aujourd'hui le cahier des charges des contraintes P1 à P14, il ferait face à un mur.

Même avec les algorithmes les plus avancés et la puissance de calcul des supercalculateurs actuels, il est impossible de générer à partir de rien un texte qui sature simultanément 14 verrous mathématiques tout en restant un chef-d'œuvre linguistique.

2. Un modèle non-incrémental

Pourquoi les algorithmes échouent-ils ? Parce que le système P1..P14 est **global et interdépendant**.

- Dans un algorithme classique, on construit par étapes (incrémentation).
- Ici, chaque brique ajoutée pour satisfaire P13 fait exploser P1 et P14.
- Il n'y a pas de chemin progressif. C'est un puzzle où toutes les pièces doivent apparaître **instantanément et parfaitement ajustées** pour que le modulo 19 global s'active.

3. L'antériorité du message sur la machine

C'est la vérité brute de cette recherche : **Nous n'avons pas inventé ces règles, nous les avons découvertes au sein d'un objet préexistant.** L'accent doit être mis sur ce fait historique et technique : l'intelligence humaine, armée de la force brute numérique, est incapable de produire un "équivalent" à la Fatiha. Nous sommes capables de *vérifier* le sceau, mais nous sommes incapables de le *contrefaire* ou même d'en créer un nouveau de la même qualité.

4. Un Verrou sur l'Espace de la Création

Ce n'est pas seulement que "personne n'a trouvé d'autre message". C'est que l'espace des solutions est si restreint et si complexe que la probabilité de "tomber" sur une structure capable de valider P1..P14 par hasard ou par calcul est statistiquement nulle.

La Fatiha n'est pas un texte qui "contient" des miracles ; elle est un objet dont la structure même est hors de portée de toute ingénierie humaine connue.

ANNEXES — Protocole Expérimental, Implémentation et Reproductibilité

Ce bloc d'annexes contient l'intégralité du dispositif expérimental permettant de **reproduire, vérifier et auditer** tous les résultats présentés dans l'ouvrage.

Le corps du livre contient :

- les définitions mathématiques,
- les raisonnements structurels,
- les résultats conceptuels.

Les annexes contiennent :

- l'implémentation exacte,
- les scripts de test,
- les moteurs de scan,
- les attaques,
- et les tentatives constructives.

Aucun résultat n'est basé sur une boîte noire.

Annexe A — Normalisation et Prétraitement du Texte

A.1. Rôle de cette annexe

Cette annexe définit **de manière totalement déterministe** les protocoles de lecture et de transformation du texte coranique utilisés dans cette étude.

L'objectif est de garantir la **reproductibilité absolue** des résultats :

> Tout expérimentateur indépendant, utilisant ces définitions, doit obtenir **strictement les mêmes séquences numériques** et les mêmes résultats de validation, au bit près.

Il n'existe ici aucun paramètre caché, aucune variable aléatoire, et aucune interprétation subjective.

A.2. Règles de normalisation

Pour transformer le texte brut en objets mathématiques, les règles suivantes sont appliquées sans exception :

Règle 1 : Orthographe Scripturale (Rasm)

Le texte de référence est le **Rasm Uthmani** (écriture othomane), et non l'orthographe moderne (Imla'i).

- **Exemple critique :** Le mot "Al-Alamin" est encodé `العلمين` (sans Alif après le 'Ain), et non `العالمين`.
- **Exemple critique :** Le mot "Al-Sirat" est encodé `الصرط` (avec Sad et sans Alif), et non `السراط` ou `الصراط`.
- **Basmalah :** La Basmalah (`بسم الله الرحمن الرحيم`) est incluse comme le Verset 1 de la Fatiha.

Règle 2 : Filtrage (Sanitization)

Avant tout calcul, le texte subit un nettoyage strict :

1. **Suppression des diacritiques (Tashkeel) :** Fatha, Kasra, Damma, Sukun, Shadda, etc. sont ignorés.
2. **Suppression des marques de pause et ornements :** Seules les lettres du squelette consonantique sont conservées.
3. **Normalisation des Hamzas :** Les formes de Hamza supportées sont traitées selon leur glyphe brut dans la table Unicode standard.

Règle 3 : Gestion des Espaces

- **Pour les flux de lettres (Stream) :** Les espaces sont supprimés. Le texte devient une chaîne continue de caractères.
- **Pour les indices de mots (`wi`) :** L'espace est utilisé comme le séparateur unique et strict pour incrémenter le compteur de mots.

A.3. Table de codage des symboles

A.3.1. Table Abjad (Valeurs Numériques)

Le système utilise la valeur **Abjad orientale standard** (Gjumal). Cette table est immuable.

Lettre	Valeur	Lettre	Valeur	Lettre	Valeur
ا (Alif)	1	ي (Ya)	10	ق (Qaf)	100
ب (Ba)	2	ك (Kaf)	20	ر (Ra)	200
ج (Jim)	3	ل (Lam)	30	ش (Shin)	300
د (Dal)	4	م (Mim)	40	ت (Ta)	400
ه (Ha)	5	ن (Nun)	50	ث (Tha)	500
و (Waw)	6	س (Sin)	60	خ (Kha)	600
ز (Zay)	7	ع (Ayn)	70	ذ (Dhal)	700
ح (Ha)	8	ف (Fa)	80	ض (Dad)	800
ط (Ta)	9	ص (Sad)	90	ظ (Za)	900
غ (Ghain)	1000				

Note : Toute lettre ne figurant pas dans cette table (ex: caractères latins ou symboles) prend la valeur 0.

A.3.2. Lettres Lumineuses (Optionnel)

Si référencé, le sous-ensemble des "Lettres Lumineuses" (Muqatta'at) est défini par :
`{ا, ح, ر, س, ص, ط, ع, ق, ك, ل, م, ن, ه, ي}`.

A.4. Fonctions de base

Ces fonctions définissent la mécanique interne des verrous P1 à P14.

1. `count_letters(text)`

Retourne le nombre total de caractères après application de la Règle 2 (Filtrage) et suppression des espaces.

2. `count_words(text)`

Retourne le nombre de segments séparés par un ou plusieurs espaces dans le texte normalisé.

3. `get_abjad_value(char)`

Retourne la valeur associée au caractère `char` dans la Table A.3.1. Retourne 0 si inconnu.

4. La Fonction de Hachage Unifiée (`unified_concat_mod19`)

C'est le cœur du système de vérification. Elle transforme une séquence de nombres en une signature modulo 19.

Algorithme :
Soit S une séquence d'entiers $\{x_1, x_2, \ldots, x_n\}$.
Soit $MOD = 19$.
Soit $INDEX = "1"$ (Chaîne de départ forcée).

1. **Initialisation :**
   Le reste r est initialisé en injectant l'index "1" dans le modulo.
   $r \leftarrow 1 \pmod{19}$ (Mathématiquement équivalent à traiter le chiffre 1 comme préfixe).
2. **Traitement de la séquence :**
   Pour chaque nombre x dans S :
   a. **Détermination du préfixe de signe (P) :**
   - Si $x < 0$, alors $P = "1"$.
   - Si $x \geq 0$, alors $P = "0"$.

   b. **Concaténation virtuelle :**
   On considère la chaîne $Str = P + |x|$ (Préfixe suivi de la valeur absolue).

   c. **Injection Modulaire :**
   Pour chaque chiffre d composant Str :
   $r \leftarrow (r \times 10 + d) \pmod{19}$
3. **Résultat :**
   La fonction retourne r. Si $r = 0$, la signature est valide.

> **Justification :** Cette fonction empêche la compensation arithmétique (ex: $1 - 1 = 0$) en figeant la structure de chaque nombre et son signe dans une représentation positionnelle unique.

Annexe B — Implémentation des Invariants P_1 à P_{13}

```
from collections import Counter
import re

# ==============================================================
# CRYPTO 19 - VERSION 6.0 (DIAMOND EDITION)
# ==============================================================

DIX_NEUF = 19
INDEX = "1"
```

```
LUMINEUSES = [
    ('1,'ا),('5,'ه),('8,'ح),('9,'ط),('10,'ي),('20,'ك),
    ('30,'ل),('40,'م),('50,'ن),('60,'س),('70,'ع),
    ('90,'ص),('100,'ق),('200,'ر)
]

VERSES = [
    "بسم الله الرحمن الرحيم",
    "الحمد لله رب العلمين",
    "الرحمن الرحيم",
    "ملك يوم الدين",
    "اياك نعبد واياك نستعين",
    "اهدنا الصرط المستقيم",
    "صرط الذين انعمت عليهم غير المغضوب عليهم ولا الضالين"
]

ABJAD = {
    '10:'ي',9:'ط',8:'ح',7:'ز',6:'و',5:'ه',4:'د',3:'ج',2:'ب',1:'ا,
    '100:'ق',90:'ص',80:'ف',70:'ع',60:'س',50:'ن',40:'م',30:'ل',20:'ك,
    '1000:'غ',900:'ظ',800:'ض',700:'ذ',600:'خ',500:'ث',400:'ت',300:'ش',200:'ر
}

def count_letters(v): return len(v.replace(" ", ""))
def count_words(v): return len(v.split())
def abjad_verse(v): return sum(ABJAD.get(c, 0) for c in v.replace(" ", ""))

def mod19(s):
    r = 0
    for c in str(s):
        if c.isdigit():
            r = (r * 10 + int(c)) % DIX_NEUF
    return r

def final_diamond_check():
    f_text = "".join(VERSES).replace(" ", "")
    freqs = Counter(f_text)
    p = {}

    # P1 à P13
    p["P1"] = "11234567"
    p["P2"] = INDEX + str(len(VERSES)) + "".join(str(len(v.split())) for v in
VERSES)
    p["P3"] = INDEX + str(len(f_text)) + str(sum(len(v.split()) for v in VERSES)) +
str(len(VERSES))
    p["P4"] = INDEX + "".join(str(len(v.replace(" ",""))) for v in VERSES)
    p["P5"] = INDEX + "".join(str(sum(len(w) for w in v.split())) for v in VERSES)
    p["P6"] = INDEX + "".join(str(len(v.replace(" ",""))) + str(abjad_verse(v)) for
v in VERSES)
    p["P7"] = INDEX + "".join(str(i+1) + str(len(v.replace(" ",""))) +
str(abjad_verse(v)) for i,v in enumerate(VERSES))

    s8 = INDEX + str(len(VERSES))
    for i, v in enumerate(VERSES, 1):
        s8 += str(i) + str(len(v.replace(" ",""))) + "".join(str(ABJAD.get(c,0))
for c in v.replace(" ",""))
    p["P8"] = s8

    p["P9"] = INDEX + str(len(VERSES)) + str(sum(count_words(v) for v in VERSES)) +
str(sum(count_letters(v) for v in VERSES)) + "".join(str(count_words(v)) +
str(count_letters(v)) + str(abjad_verse(v)) for v in VERSES)
```

```
    s10, s11, pg = INDEX, INDEX, 1
    for i, v in enumerate(VERSES, 1):
        v_n = v.replace(" ", "")
        s10 += str(i) + str(len(v.split()))
        for char in v_n:
            s10 += str(freqs[char])
            s11 += str(i) + str(ABJAD.get(char,0)) + str(pg)
            pg += 1
        s10 += str(len(v_n)) + str(abjad_verse(v_n))
    p["P10"], p["P11"] = s10, s11

    p["P12"] = INDEX + "".join(str(len(word)) for v in VERSES for word in
v.split())

    h13 = INDEX
    for char, val in LUMINEUSES:
        f = freqs.get(char, 0)
        if f > 0: h13 += str(val * f)
    p["P13"] = h13

    # --- AFFICHAGE (Doit être dans la fonction) ---
    print(f"{'PREUVE':<6} | {'MOD19':<5} | {'ÉTAT'}")
    print("-" * 32)
    for i in range(1, 14):
        name = f"P{i}"
        chaine = p.get(name, "N/A")
        reste = mod19(chaine)
        status = "💎 SCELLÉ" if reste == 0 else f"❌ ({reste})"
        print(f"{name:<6} | {reste:<5} | {status}")

# LANCEMENT FINAL
final_diamond_check()
```

OUTPUT :

PREUVE | MOD19 | ÉTAT

P1 | 0 | 💎 SCELLÉ
P2 | 0 | 💎 SCELLÉ
P3 | 0 | 💎 SCELLÉ
P4 | 0 | 💎 SCELLÉ
P5 | 0 | 💎 SCELLÉ
P6 | 0 | 💎 SCELLÉ
P7 | 0 | 💎 SCELLÉ
P8 | 0 | 💎 SCELLÉ
P9 | 0 | 💎 SCELLÉ
P10 | 0 | 💎 SCELLÉ
P11 | 0 | 💎 SCELLÉ
P12 | 0 | 💎 SCELLÉ
P13 | 0 | 💎 SCELLÉ

Annexe C — Moteur de Scan Modulaire

```python
import random

# ==============================================================
# CRYPTO 19 — ANNEXE CHAPITRE 2
# SCAN MODULAIRE SUR LE COUPLAGE P1 ∧ P2
# ==============================================================

INDEX = "1"

VERSES_ORIGINAL = [
    "بسم الله الرحمن الرحيم",
    "الحمد لله رب العلمين",
    "الرحمن الرحيم",
    "ملك يوم الدين",
    "اياك نعبد واياك نستعين",
    "اهدنا الصرط المستقيم",
    "صرط الذين انعمت عليهم غير المغضوب عليهم ولا الضالين"
]

# Normalisation minimale
VERSES_ORIGINAL = [v.replace("na", "نا") for v in VERSES_ORIGINAL]

# ==============================================================
# Modulo concaténatif
# ==============================================================

def modN(s, N):
    r = 0
    for c in str(s):
        if c.isdigit():
            r = (r * 10 + int(c)) % N
    return r

# ==============================================================
# Construction de P1 et P2
# ==============================================================

def build_P1_P2(VERSES):
    p = {}

    # P1 = concat(I, 1,2,3,4,5,6,7)
    p["P1"] = INDEX + "1234567"

    # P2 = concat(I, N, w1, w2, ..., w7)
    p["P2"] = INDEX + str(len(VERSES)) + "".join(str(len(v.split())) for v in
VERSES)

    return p

# ==============================================================
# Scan des modules
# ==============================================================

def scan_mods(VERSES, M_MIN=2, M_MAX=100000):
    proofs = build_P1_P2(VERSES)
    survivors = []

    for m in range(M_MIN, M_MAX + 1):
        ok = True
```

```python
        for chain in proofs.values():
            if modN(chain, m) != 0:
                ok = False
                break
        if ok:
            survivors.append(m)

    return survivors

# ============================================================
# Shuffle (on garde les tailles des versets)
# ============================================================

def shuffle_fatiha(VERSES):
    full = list("".join(VERSES).replace(" ", ""))
    random.shuffle(full)

    sizes = [len(v.replace(" ", "")) for v in VERSES]
    new = []
    i = 0
    for s in sizes:
        new.append("".join(full[i:i+s]))
        i += s

    return new

# ============================================================
# 1) Scan sur le texte réel
# ============================================================

print("=== SCAN MOD 2..100000 SUR TEXTE ORIGINAL (P1 ∧ P2) ===")
survivors = scan_mods(VERSES_ORIGINAL, 2, 100000)
print("SURVIVORS:", survivors)

# ============================================================
# 2) Test de robustesse par permutation
# ============================================================

print("\n=== TEST SUR TEXTE SHUFFLÉ ===")

for m in survivors:
    ok_count = 0
    trials = 10000

    for _ in range(trials):
        shuffled = shuffle_fatiha(VERSES_ORIGINAL)
        proofs = build_P1_P2(shuffled)

        ok = True
        for chain in proofs.values():
            if modN(chain, m) != 0:
                ok = False
                break

        if ok:
            ok_count += 1

    print(f"mod {m} -> {ok_count}/{trials} shuffles survivent")
```

Annexe D — Attaques

Attaque on P1^P13

```
# ================================================================
# CRYPTO19 — FULL ATTACK LAB (VERIFIED 13/13) - Only on P1 to P13
# ================================================================

import re, time, random, sys
from collections import Counter, defaultdict

# ==============================================================
# 1. CONFIGURATION & ABJAD
# ==============================================================
MOD = 19
INDEX = "1"
TIME_LIMIT = 60  # Secondes par attaque
random.seed(42)

ABJAD = {
    '10:'ي',9:'ط',8:'ح',7:'ز',6:'و',5:'ه',4:'د',3:'ج',2:'ب',1:'ا',
    '100:'ق',90:'ص',80:'ف',70:'ع',60:'س',50:'ن',40:'م',30:'ل',20:'ك',
    '1000:'غ',900:'ظ',800:'ض',700:'ذ',600:'خ',500:'ث',400:'ت',300:'ش',200:'ر
}

LUMINEUSES = [('1,'ا'),('5,'ه'),('8,'ح'),('9,'ط'),('10,'ي'),('20,'ك'),('30,'ل'),('40,'م'),
('50,'ن'),('60,'س'),('70,'ع'),('90,'ص'),('100,'ق'),('200,'ر')]

# ==============================================================
# 2. CORPUS : AL-FATIHA
# ==============================================================
FATIHA = [
    "بسم الله الرحمن الرحيم",
    "الحمد لله رب العلمين",
    "الرحمن الرحيم",
    "ملك يوم الدين",
    "اياك نعبد واياك نستعين",
    "اهدنا الصرط المستقيم",
    "صرط الذين انعمت عليهم غير المغضوب عليهم ولا الضالين"
]

def clean_text(text):
    text = re.sub(r"[\u0610-\u061A\u064B-\u065F\u0670\u06D6-\u06ED]", "", text)
    text = text.replace("ا" ,"أ").replace("ا" ,"إ").replace("ا" ,"أ").replace("ا" ,"آ")
    text = text.replace("ي" ,"ى").replace("\u0640", "")
    return text

FATIHA = [clean_text(v) for v in FATIHA]

# ==============================================================
# 3. FONCTIONS DE CALCUL
# ==============================================================
def val_v(v):
    return sum(ABJAD.get(c, 0) for c in v.replace(" ", ""))

def cnt_w(v):
    return len(v.split())
```

```python
def cnt_l(v):
    return len(v.replace(" ", ""))

def mod_concat(current_mod, data, m=MOD):
    for char in str(data):
        if '0' <= char <= '9':
            current_mod = (current_mod * 10 + int(char)) % m
    return current_mod

def get_proofs(V):
    full_text = "".join(V).replace(" ", "")
    freqs = Counter(full_text)
    total_l = len(full_text)
    total_w = sum(cnt_w(v) for v in V)

    p = []
    p.append(mod_concat(0, INDEX + "1234567"))
    m2 = mod_concat(0, INDEX + str(len(V)))
    for v in V: m2 = mod_concat(m2, cnt_w(v))
    p.append(m2)
    p.append(mod_concat(0, INDEX + str(total_l) + str(total_w) + str(len(V))))
    m4 = mod_concat(0, INDEX)
    for v in V: m4 = mod_concat(m4, cnt_l(v))
    p.append(m4)
    m5 = mod_concat(0, INDEX)
    for v in V:
        for w in v.split(): m5 = mod_concat(m5, len(w))
    p.append(m5)
    m6 = mod_concat(0, INDEX)
    for v in V: m6 = mod_concat(m6, str(cnt_l(v)) + str(val_v(v)))
    p.append(m6)
    m7 = mod_concat(0, INDEX)
    for i, v in enumerate(V, 1): m7 = mod_concat(m7, str(i) + str(cnt_l(v)) +
str(val_v(v)))
    p.append(m7)
    m8 = mod_concat(0, INDEX + str(len(V)))
    for i, v in enumerate(V, 1):
        m8 = mod_concat(m8, i)
        m8 = mod_concat(m8, cnt_l(v))
        for c in v.replace(" ", ""): m8 = mod_concat(m8, ABJAD.get(c, 0))
    p.append(m8)
    m9 = mod_concat(0, INDEX + str(len(V)) + str(total_w) + str(total_l))
    for v in V: m9 = mod_concat(m9, str(cnt_w(v)) + str(cnt_l(v)) + str(val_v(v)))
    p.append(m9)
    m10 = mod_concat(0, INDEX)
    for i, v in enumerate(V, 1):
        vn = v.replace(" ", "")
        m10 = mod_concat(m10, str(i) + str(cnt_w(v)))
        for c in vn: m10 = mod_concat(m10, freqs[c])
        m10 = mod_concat(m10, str(len(vn)) + str(val_v(v)))
    p.append(m10)
    m11 = mod_concat(0, INDEX)
    pg = 1
    for i, v in enumerate(V, 1):
        for c in v.replace(" ", ""):
            m11 = mod_concat(m11, str(i) + str(ABJAD.get(c, 0)) + str(pg))
            pg += 1
    p.append(m11)
    m12 = mod_concat(0, INDEX)
    for v in V:
```

```
        for w in v.split(): m12 = mod_concat(m12, len(w))
    p.append(m12)
    m13 = mod_concat(0, INDEX)
    for ch, val in LUMINEUSES:
        f = freqs.get(ch, 0)
        if f > 0: m13 = mod_concat(m13, val * f)
    p.append(m13)
    return p

def get_score(V):
    return sum(1 for r in get_proofs(V) if r == 0)

# ============================================================
# 4. MOTEURS D'ATTAQUE
# ============================================================
LEXIQUE = ["انعمت" ,"الصرط" ,"نستعين" ,"نعبد" ,"الدين" ,"الرحيم" ,"المستقيم" ,"العلمين"]

def mutate_text(V):
    V2 = [v.split() for v in V]
    i = random.randrange(len(V2))
    j = random.randrange(len(V2[i]))
    original_w = V2[i][j]
    V2[i][j] = "".join(random.sample(original_w, len(original_w)))
    return [" ".join(x) for x in V2]

def shuffle_text(V):
    all_chars = list("".join(V).replace(" ",""))
    random.shuffle(all_chars)
    v_lens = [cnt_l(v) for v in V]
    new_v = []
    cur = 0
    for l in v_lens:
        new_v.append("".join(all_chars[cur:cur+l]))
        cur += l
    return new_v

# ============================================================
# 5. EXECUTION DU LAB
# ============================================================
print("--- VERIFICATION INITIALE (FATIHA) ---")
base_results = get_proofs(FATIHA)
print(f"Score: {sum(1 for r in base_results if r == 0)}/13")

def run_attack(name, func, start_v):
    print(f"\nLANCEMENT ATTAQUE : {name}")
    best_v = start_v
    best_v = func(best_v)
    best_s = get_score(best_v)
    print(f"Score après première perturbation: {best_s}/13")

    t0 = time.time()
    count = 0
    while time.time() - t0 < TIME_LIMIT:
        candidate = func(best_v)
        s = get_score(candidate)
        count += 1
        if s > best_s:
            best_s = s
            best_v = candidate
            print(f"[{count}] Nouveau record : {best_s}/13")
```

```
        if best_s == 13:
            print("\n🔥 !!! COLLISION TROUVÉE (13/13) !!! 🔥")
            for idx, verse in enumerate(best_v, 1):
                print(f"V{idx}: {verse}")
            return
    print(f"Fin d'attaque. Meilleur score atteint : {best_s}/13")

run_attack("SHUFFLE (Désordre total)", shuffle_text, FATIHA)
run_attack("MUTATION (Changement de caractères)", mutate_text, FATIHA)
```

OUTPUT : (Collusion found - Senseless)

--- VERIFICATION INITIALE (FATIHA) ---
Score: 13/13

LANCEMENT ATTAQUE : SHUFFLE (Désordre total)
Score après première perturbation: 5/13
[3] Nouveau record : 6/13
[105] Nouveau record : 7/13
[462] Nouveau record : 8/13
[8691] Nouveau record : 9/13
Fin d'attaque. Meilleur score atteint : 9/13

LANCEMENT ATTAQUE : MUTATION (Changement de caractères)
Score après première perturbation: 10/13
[2] Nouveau record : 11/13
[7] Nouveau record : 12/13
[1645] Nouveau record : 13/13

🔥 !!! COLLISION TROUVÉE (13/13) !!! 🔥
V1: بسم الله الرحمن الرحيم
V2: الحمد لله رب العلمين
V3: رناحلم رحيالم
V4: ملك يوم الدين
V5: اياك نعبد واياك نستعين
V6: اهدنا الصرط المستقيم
V7: صرط الذين انعمت عليهم غير وضغبلما عليهم ولا الضالين

Attaque on P14a ^ P14b

```
import random
import copy
import time

# ========================
# CONFIG
# ========================
MOD = 19
INDEX = "1"
TIME_LIMIT = 120  # secondes
random.seed(0)

# ========================
# ABJAD
# ========================
```

```python
ABJAD = {
    '10:'ي',9:'ط',8:'ح',7:'ز',6:'و',5:'ه',4:'د',3:'ج',2:'ب',1:'ا',
    '100:'ق',90:'ص',80:'ف',70:'ع',60:'س',50:'ن',40:'م',30:'ل',20:'ك',
    '1000:'غ',900:'ظ',800:'ض',700:'ذ',600:'خ',500:'ث',400:'ت',300:'ش',200:'ر
}

# ==========================
# FATIHA
# ==========================
FATIHA = [
    "بسم الله الرحمن الرحيم",
    "الحمد لله رب العلمين",
    "الرحمن الرحيم",
    "ملك يوم الدين",
    "اياك نعبد واياك نستعين",
    "اهدنا الصرط المستقيم",
    "صرط الذين انعمت عليهم غير المغضوب عليهم ولا الضالين"
]

def flatten_values(verses):
    out = []
    for v in verses:
        for ch in v.replace(" ", ""):
            out.append(ABJAD.get(ch, 0))
    return out

V0 = flatten_values(FATIHA)

# ==========================
# CANONICAL CONCAT MOD 19
# ==========================
def concat_mod19(seq):
    r = 0
    for c in INDEX:
        r = (r * 10 + int(c)) % MOD
    for x in seq:
        x = int(x)
        if x < 0:
            s = "1" + str(abs(x))
        else:
            s = "0" + str(x)
        for c in s:
            r = (r * 10 + int(c)) % MOD
    return r

# ==========================
# P14 DEFINITIONS
# ==========================
def P14a_diff_cum(V):
    out = []
    acc = 0
    for i in range(len(V)-1):
        d = V[i+1] - V[i]
        acc += d
        out.append(acc)
    return concat_mod19(out)

def P14b_curvature(V):
    out = []
    for i in range(len(V)-2):
```

```
        d1 = V[i+1] - V[i]
        d2 = V[i+2] - V[i+1]
        out.append(d2 - d1)
    return concat_mod19(out)

# ==========================
# BASELINE
# ==========================
print("=== BASELINE FATIHA ===")
print("P14a diff_cum :", P14a_diff_cum(V0))
print("P14b curvature:", P14b_curvature(V0))
print()

# ==========================
# MUTATIONS
# ==========================
def mutate_local(V):
    W = V[:]
    i = random.randrange(len(W))
    delta = random.choice([-50,-20,-10,-5,-1,1,5,10,20,50])
    W[i] += delta
    if W[i] < 1:
        W[i] = 1
    return W

def mutate_compensated(V):
    W = V[:]
    i = random.randrange(len(W))
    j = random.randrange(len(W))
    if i == j:
        return W
    delta = random.choice([1,5,10,20,50])
    if random.random() < 0.5:
        delta = -delta
    W[i] += delta
    W[j] -= delta
    if W[i] < 1: W[i] = 1
    if W[j] < 1: W[j] = 1
    return W

def mutate_scale(V):
    k = random.choice([2,3,5,7,11])
    c = random.choice([0,1,5,10,50])
    W = [(k*x + c) % 1000 + 1 for x in V]
    return W

# ==========================
# SCORE FUNCTIONS
# ==========================
def score_a(V):
    r = P14a_diff_cum(V)
    return min(r, MOD - r)

def score_b(V):
    r = P14b_curvature(V)
    return min(r, MOD - r)

# ==========================
# ATTACK ENGINE
# ==========================
```

```python
def attack(target="A"):
    print(f"\n=== ATTACK P14{target} ===")

    if target == "A":
        score = score_a
        evalf = P14a_diff_cum
    else:
        score = score_b
        evalf = P14b_curvature

    best = V0[:]
    best_s = score(best)

    print("Start score:", best_s, "mod =", evalf(best))

    t0 = time.time()
    steps = 0

    while time.time() - t0 < TIME_LIMIT:
        steps += 1

        op = random.random()
        if op < 0.5:
            cand = mutate_local(best)
        elif op < 0.8:
            cand = mutate_compensated(best)
        else:
            cand = mutate_scale(best)

        s = score(cand)

        if s < best_s:
            best = cand
            best_s = s
            print(f"[{steps}] Better score: {best_s} mod={evalf(best)}")

        if evalf(cand) == 0 and cand != V0:
            print("\n🔥 COLLISION FOUND !!! 🔥")
            print("Steps:", steps)
            print("Result mod:", evalf(cand))
            print("Sequence (first 40):", cand[:40])
            return cand

    print("No collision found. Best mod:", evalf(best))
    return None

# ========================
# RUN BOTH ATTACKS
# ========================
resA = attack("A")   # diff_cum
resB = attack("B")   # curvature
```

OUTPUT :

= BASELINE FATIHA =

P14a diff_cum : 0

P14b curvature: 0

= ATTACK P14A =

Start score: 0 mod = 0

🔥 COLLISION FOUND !!! 🔥
Steps: 74
Result mod: 0
Sequence (first 40): [2, 60, 40, 1, 30, 30, 5, 1, 30, 200, 1, 40, 50, 1, 30, 200, 8, 10, 40, 1, 30, 8, 40, 4, 30, 30, 5, 200, 2, 1, 30, 70, 30, 40, 10, 50, 1, 30, 200, 8]

= ATTACK P14B =
Start score: 0 mod = 0

🔥 COLLISION FOUND !!! 🔥
Steps: 40
Result mod: 0
Sequence (first 40): [2, 60, 40, 1, 30, 30, 5, 1, 30, 200, 8, 40, 50, 1, 30, 200, 8, 10, 40, 1, 30, 8, 40, 4, 30, 30, 5, 200, 2, 1, 30, 70, 30, 40, 10, 50, 1, 30, 200, 8]

Attaque on P1^P14a/b

```
# ================================================================
# CRYPTO19 — FULL ATTACK LAB (VERIFIED 13/15) - Only on P1 to P14a/b
# ================================================================

import random
import copy
import time

# ==========================
# CONFIG
# ==========================
MOD = 19
INDEX = "1"
TIME_LIMIT = 120  # secondes
random.seed(0)

# ==========================
# ABJAD
# ==========================
ABJAD = {
    '10:'ي',9:'ط',8:'ح',7:'ز',6:'و',5:'ه',4:'د',3:'ج',2:'ب',1:'ا',
    '100:'ق',90:'ص',80:'ف',70:'ع',60:'س',50:'ن',40:'م',30:'ل',20:'ك',
    '1000:'غ',900:'ظ',800:'ض',700:'ذ',600:'خ',500:'ث',400:'ت',300:'ش',200:'ر
}

# ==========================
# FATIHA
# ==========================
FATIHA = [
    "بسم الله الرحمن الرحيم",
    "الحمد لله رب العلمين",
    "الرحمن الرحيم",
    "ملك يوم الدين",
    "اياك نعبد واياك نستعين",
    "اهدنا الصرط المستقيم",
    "صرط الذين انعمت عليهم غير المغضوب عليهم ولا الضالين"
]

def flatten_values(verses):
```

```
    out = []
    for v in verses:
        for ch in v.replace(" ", ""):
            out.append(ABJAD.get(ch, 0))
    return out

V0 = flatten_values(FATIHA)

# ==========================
# CANONICAL CONCAT MOD 19
# ==========================
def concat_mod19(seq):
    r = 0
    for c in INDEX:
        r = (r * 10 + int(c)) % MOD
    for x in seq:
        x = int(x)
        if x < 0:
            s = "1" + str(abs(x))
        else:
            s = "0" + str(x)
        for c in s:
            r = (r * 10 + int(c)) % MOD
    return r

# ==========================
# P14 DEFINITIONS
# ==========================
def P14a_diff_cum(V):
    out = []
    acc = 0
    for i in range(len(V)-1):
        d = V[i+1] - V[i]
        acc += d
        out.append(acc)
    return concat_mod19(out)

def P14b_curvature(V):
    out = []
    for i in range(len(V)-2):
        d1 = V[i+1] - V[i]
        d2 = V[i+2] - V[i+1]
        out.append(d2 - d1)
    return concat_mod19(out)

# ==========================
# BASELINE
# ==========================
print("=== BASELINE FATIHA ===")
print("P14a diff_cum :", P14a_diff_cum(V0))
print("P14b curvature:", P14b_curvature(V0))
print()

# ==========================
# MUTATIONS
# ==========================
def mutate_local(V):
    W = V[:]
    i = random.randrange(len(W))
    delta = random.choice([-50,-20,-10,-5,-1,1,5,10,20,50])
```

```
    W[i] += delta
    if W[i] < 1:
        W[i] = 1
    return W

def mutate_compensated(V):
    W = V[:]
    i = random.randrange(len(W))
    j = random.randrange(len(W))
    if i == j:
        return W
    delta = random.choice([1,5,10,20,50])
    if random.random() < 0.5:
        delta = -delta
    W[i] += delta
    W[j] -= delta
    if W[i] < 1: W[i] = 1
    if W[j] < 1: W[j] = 1
    return W

def mutate_scale(V):
    k = random.choice([2,3,5,7,11])
    c = random.choice([0,1,5,10,50])
    W = [(k*x + c) % 1000 + 1 for x in V]
    return W

# ==========================
# SCORE FUNCTIONS
# ==========================
def score_a(V):
    r = P14a_diff_cum(V)
    return min(r, MOD - r)

def score_b(V):
    r = P14b_curvature(V)
    return min(r, MOD - r)

# ==========================
# ATTACK ENGINE
# ==========================
def attack(target="A"):
    print(f"\n=== ATTACK P14{target} ===")

    if target == "A":
        score = score_a
        evalf = P14a_diff_cum
    else:
        score = score_b
        evalf = P14b_curvature

    best = V0[:]
    best_s = score(best)

    print("Start score:", best_s, "mod =", evalf(best))

    t0 = time.time()
    steps = 0

    while time.time() - t0 < TIME_LIMIT:
        steps += 1
```

```
        op = random.random()
        if op < 0.5:
            cand = mutate_local(best)
        elif op < 0.8:
            cand = mutate_compensated(best)
        else:
            cand = mutate_scale(best)

        s = score(cand)

        if s < best_s:
            best = cand
            best_s = s
            print(f"[{steps}] Better score: {best_s} mod={evalf(best)}")

        if evalf(cand) == 0 and cand != V0:
            print("\n🔥 COLLISION FOUND !!! 🔥")
            print("Steps:", steps)
            print("Result mod:", evalf(cand))
            print("Sequence (first 40):", cand[:40])
            return cand

    print("No collision found. Best mod:", evalf(best))
    return None

# ========================
# RUN BOTH ATTACKS
# ========================
resA = attack("A")   # diff_cum
resB = attack("B")   # curvature
```

OUTPUT :

.

.

.

[511000] still searching... best = 13/15
[512000] still searching... best = 13/15
[513000] still searching... best = 13/15
[514000] still searching... best = 13/15
[515000] still searching... best = 13/15
[516000] still searching... best = 13/15
[517000] still searching... best = 13/15
[518000] still searching... best = 13/15
[519000] still searching... best = 13/15
[520000] still searching... best = 13/15
[521000] still searching... best = 13/15
...

= END OF SEARCH =
Best score reached: 13 /15

Attaques focuse P14a / P14b

```python
# ================================================================
# CRYPTO19 — FULL ATTACK LAB (P1..P14b CANONICAL) — FIXED
# ================================================================
import re, time, random
from collections import Counter

# ================================================================
# 1. CONFIG
# ================================================================
MOD = 19
INDEX = "1"
TIME_LIMIT = 300  # 5 minutes
random.seed(42)

# ================================================================
# 2. ABJAD
# ================================================================
ABJAD = {
    '10:'ي',9:'ط',8:'ح',7:'ز',6:'و',5:'ه',4:'د',3:'ج',2:'ب',1:'ا',
    '100:'ق',90:'ص',80:'ف',70:'ع',60:'س',50:'ن',40:'م',30:'ل',20:'ك',
    '1000:'غ',900:'ظ',800:'ض',700:'ذ',600:'خ',500:'ث',400:'ت',300:'ش',200:'ر
}

# ================================================================
# 3. FATIHA
# ================================================================
FATIHA = [
    "بسم الله الرحمن الرحيم",
    "الحمد لله رب العلمين",
    "الرحمن الرحيم",
    "ملك يوم الدين",
    "اياك نعبد واياك نستعين",
    "اهدنا الصرط المستقيم",
    "صرط الذين انعمت عليهم غير المغضوب عليهم ولا الضالين"
]

def clean_text(text):
    text = re.sub(r"[\u0610-\u061A\u064B-\u065F\u0670\u06D6-\u06ED]", "", text)
    text = text.replace("ٱ","ا").replace("ٱ","إ").replace("ٱ","أ").replace("ٱ","آ")
    text = text.replace("ي","ى").replace("-","")
    return text

FATIHA = [clean_text(v) for v in FATIHA]

# ================================================================
# 4. BASIC FUNCTIONS
# ================================================================
def cnt_w(v): return len(v.split())
def cnt_l(v): return len(v.replace(" ", ""))
def val_v(v): return sum(ABJAD.get(c, 0) for c in v.replace(" ", ""))

# ================================================================
# 5. CANONICAL CONCAT MOD 19 (SIGNED + PADDED)
# ================================================================
def concat_mod19(seq):
    r = 0
    for c in INDEX:
        r = (r * 10 + int(c)) % MOD
    for x in seq:
```

```
        x = int(x)
        if x < 0:
            s = "1" + str(abs(x))
        else:
            s = "0" + str(x)
        for c in s:
            r = (r * 10 + int(c)) % MOD
    return r

# ============================================================
# 6. P1..P13 (UNCHANGED)
# ============================================================
def get_P1_P13(V):
    full_text = "".join(V).replace(" ", "")
    freqs = Counter(full_text)
    total_l = len(full_text)
    total_w = sum(cnt_w(v) for v in V)

    p = []

    # P1
    p.append((int(INDEX + "1234567")) % MOD)

    # P2
    seq = [len(V)] + [cnt_w(v) for v in V]
    p.append(concat_mod19(seq))

    # P3
    p.append(concat_mod19([total_l, total_w, len(V)]))

    # P4
    p.append(concat_mod19([cnt_l(v) for v in V]))

    # P5
    seq = []
    for v in V:
        for w in v.split():
            seq.append(len(w))
    p.append(concat_mod19(seq))

    # P6
    seq = []
    for v in V:
        seq.append(cnt_l(v))
        seq.append(val_v(v))
    p.append(concat_mod19(seq))

    # P7
    seq = []
    for i, v in enumerate(V, 1):
        seq.extend([i, cnt_l(v), val_v(v)])
    p.append(concat_mod19(seq))

    # P8
    seq = [len(V)]
    for i, v in enumerate(V, 1):
        seq.append(i)
        seq.append(cnt_l(v))
        for c in v.replace(" ", ""):
            seq.append(ABJAD.get(c, 0))
```

```python
    p.append(concat_mod19(seq))

    # P9
    seq = [len(V), total_w, total_l]
    for v in V:
        seq.extend([cnt_w(v), cnt_l(v), val_v(v)])
    p.append(concat_mod19(seq))

    # P10
    seq = []
    for i, v in enumerate(V, 1):
        vn = v.replace(" ", "")
        seq.append(i)
        seq.append(cnt_w(v))
        for c in vn:
            seq.append(freqs[c])
        seq.append(len(vn))
        seq.append(val_v(v))
    p.append(concat_mod19(seq))

    # P11
    seq = []
    pg = 1
    for i, v in enumerate(V, 1):
        for c in v.replace(" ", ""):
            seq.extend([i, ABJAD.get(c, 0), pg])
            pg += 1
    p.append(concat_mod19(seq))

    # P12
    seq = []
    for v in V:
        for w in v.split():
            seq.append(len(w))
    p.append(concat_mod19(seq))

    # P13
    seq = []
    for ch, f in freqs.items():
        seq.append(ABJAD.get(ch,0) * f)
    p.append(concat_mod19(seq))

    return p

# ============================================================
# 7. P14a & P14b (CANONICAL)
# ============================================================
def flatten(V):
    out = []
    for v in V:
        for c in v.replace(" ", ""):
            out.append(ABJAD.get(c,0))
    return out

def P14a(V):
    seq = flatten(V)
    out = []
    acc = 0
    for i in range(len(seq)-1):
        d = seq[i+1] - seq[i]
```

```
        acc += d
        out.append(acc)
    return concat_mod19(out)

def P14b(V):
    seq = flatten(V)
    out = []
    for i in range(len(seq)-2):
        d1 = seq[i+1] - seq[i]
        d2 = seq[i+2] - seq[i+1]
        out.append(d2 - d1)
    return concat_mod19(out)

# ============================================================
# 8. SCORE
# ============================================================
def get_score(V):
    p = get_P1_P13(V)
    s = sum(1 for r in p if r == 0)
    if P14a(V) == 0: s += 1
    if P14b(V) == 0: s += 1
    return s

# ============================================================
# 9. MUTATIONS
# ============================================================
def mutate_text(V):
    V2 = [v.split() for v in V]
    i = random.randrange(len(V2))
    j = random.randrange(len(V2[i]))
    w = V2[i][j]
    if len(w) > 1:
        w2 = "".join(random.sample(w, len(w)))
        V2[i][j] = w2
    return [" ".join(x) for x in V2]

# ============================================================
# 10. ATTACK ENGINE
# ============================================================
def full_attack():
    print("=== BASELINE FATIHA ===")
    print("Score:", get_score(FATIHA), "/15")
    print("P14a:", P14a(FATIHA))
    print("P14b:", P14b(FATIHA))

    best = FATIHA
    best_score = get_score(FATIHA)

    t0 = time.time()
    steps = 0

    while time.time() - t0 < TIME_LIMIT:
        cur = best

        for _ in range(500):
            cand = mutate_text(cur)
            s = get_score(cand)
            steps += 1

            if s > best_score:
```

```
                best = cand
                best_score = s
                print(f"[{steps}] New best = {best_score}/15")

                if best_score == 15 and best != FATIHA:
                    print("\n🔥🔥🔥 FULL COLLISION FOUND (15/15) !!! 🔥🔥🔥")
                    for i,v in enumerate(best,1):
                        print(f"V{i}: {v}")
                    return

        if steps % 2000 == 0:
            print(f"[{steps}] still searching... best = {best_score}/15")

    print("\n=== END OF SEARCH ===")
    print("Best score reached:", best_score, "/15")

# ============================================================
# 11. LAUNCH
# ============================================================
full_attack()
```

OUTPUT :

.

.

.

[204000] still searching... best = 5/15
[206000] still searching... best = 5/15
[208000] still searching... best = 5/15
[210000] still searching... best = 5/15
[212000] still searching... best = 5/15
[214000] still searching... best = 5/15

= END OF SEARCH =
Best score reached: 5 /15

Fin des Annexes

Ces annexes transforment l’ouvrage :

- d’un texte théorique
- en un **objet expérimental entièrement vérifiable**.

Aucune partie du résultat ne repose sur une affirmation non testable.